The Fabius Function in Lean

A source-level walkthrough of the `FabiusFunction` development

Architecture, exact arithmetic, analytic construction, paper facades, Thue–Morse identities, Legendre expansions, and the corrected asymptotic tower

Repository	`VladimirReshetnikov/ProveIt`
Subtree	`Analysis/FabiusFunction`
Pinned snapshot	`40c0d637d310a9f16d96f53d287438b7d9b1f53f`
Lean / mathlib	Lean `v4.32.0`; mathlib `v4.32.0`
Prepared	24 August 2026

Prepared for Vladimir Reshetnikov

Abstract

This document is a long-form walkthrough of the Lean 4 development under `Analysis/FabiusFunction` in the `VladimirReshetnikov/ProveIt` repository, pinned to commit `40c0d637d310a9f16d96f53d287438b7d9b1f53f`. Its purpose is not merely to restate the mathematics of the Fabius function. It explains how the formalization is *engineered*: which coordinate systems are used, how exact rational arithmetic is separated from real and complex analysis, where generic lemmas are factored out, how executable dyadic evaluation is certified, why the library contains two distinct uniqueness proofs, how the two Arias de Reyna papers are exposed through facade modules, and how the corrected sharp and all-orders asymptotic theories are decomposed into tractable proof obligations.

The central architectural fact is that the library does not treat the Fabius function as one definition surrounded by a flat list of lemmas. It builds a network of mutually reinforcing models:

- a bounded cumulative function on $[0, 1]$;
- Rvachev's compactly supported smooth *up* function on $[-1, 1]$;
- a signed global Thue–Morse extension on the real line;
- exact rational moment sequences and dyadic values;
- formal power series and Fourier/Laplace transforms;
- probability measures, spline and step approximants, and asymptotic saddle-point models.

The walkthrough follows the dependency graph rather than filename order. It also records the proof-engineering patterns that recur throughout the code: subtype invariants, local finiteness, strong induction over binary structure, formal-series coefficient extraction, normalization before divisibility, Schwartz-space packaging, finite-row Toeplitz arguments, and explicit separation of exact identities from asymptotic estimates.

Dependency seam

The repository's audit document records its most recent scripted scan as 141 Lean source files, 4,632 constants, and 4,139 theorems, with no project-specific axioms and no `sorry`. The pinned source tree used for this walkthrough has subsequently grown to 164 leaf modules plus the root umbrella module; Appendix A follows that actual tree. The older declaration counts are therefore quoted only as audit metadata, not as current recounts. The logical dependencies reported by the audit are the standard classical principles `propext`, `Classical.choice`, and `Quot.sound` inherited from Lean/mathlib.

How to use this walkthrough

The chapters are arranged in four passes.

Pass I: the semantic core (Chapters 1–7).
Start here to understand the four models of the function, the central `IsFabius` predicate, the exact arithmetic layer, the differential bootstrap, the contraction construction, and the global signed extension.

Pass II: exact evaluation and the two papers (Chapters 8–13).
This is the route for the executable dyadic evaluator, moment arithmetic, Fourier/probability representations, the two paper-level APIs, and the Thue–Morse/q-binomial formulas.

Pass III: approximation and asymptotics (Chapters 14–18).
These chapters explain the Legendre branch, the discrete-limit and spline branch, and the corrected asymptotic tower from the negative Laplace product to the exact Lambert phase and all-orders saddle expansion.

Pass IV: engineering and reference (Chapters 19–20 and appendices).
Use this when extending the code. It catalogues common Lean proof patterns, gives task-oriented reading paths, maps paper claims to modules, and lists every source file by role.

Code excerpts are intentionally small and structural. They preserve Lean names and theorem shapes, while eliding proof bodies when the surrounding prose explains the argument. A path beginning with `FabiusFunction.` means a module under `Analysis/FabiusFunction/Lean/FabiusFunction/`. The root facade is `Analysis/FabiusFunction/Lean/FabiusFunction.lean`.

Contents

1

Repository boundary, build, and trust surface

1.1 The focused Lean library

The Fabius development is a named Lean library embedded in the larger `ProveIt` repository. The relevant stanza of `lakefile.toml` is conceptually minimal:

```
[[lean_lib]]
name = "FabiusFunction"
srcDir = "Analysis/FabiusFunction/Lean"
```

The toolchain file pins Lean to `leanprover/lean4:v4.32.0`, and the mathlib dependency is pinned to the matching `v4.32.0`. From the repository root, the focused build command is therefore:

```
lake update
lake build FabiusFunction
```

The first command is normally needed only after cloning or after dependency changes. The second is the important boundary: it elaborates every module reachable from the library roots, not merely the paper facades a reader happens to import in an experiment.

1.2 Pinning the prose to a source snapshot

Lean code evolves quickly enough that a walkthrough written against the moving `main` branch becomes ambiguous. Every declaration and dependency claim in this document refers to commit `40c0d637d310a9f16d96f53d287438b7d9b1f53f`. In a local checkout, the reproducible sequence is:

```
git checkout 40c0d637d310a9f16d96f53d287438b7d9b1f53f
lake build FabiusFunction
```

The commit is especially relevant here because the subtree is under active mathematical development. The public facade already includes not only the two formalized papers but newer exact q-binomial formulas, Legendre results, a claim audit of a local Thue–Morse draft, and a corrected asymptotic theory.

1.3 The root facade is a curated API, not a dump

The file `Lean/FabiusFunction.lean` is short. Its importance lies in what it imports:

```
import FabiusFunction.Paper05442
import FabiusFunction.Paper06487
import FabiusFunction.PaperFabiusAsymptotic
import FabiusFunction.PaperKFoldThueMorse
import FabiusFunction.NegativeLaplace
import FabiusFunction.PeriodicCorrection
import FabiusFunction.MellinBose
import FabiusFunction.MellinFinitePart
import FabiusFunction.BoseFinitePartIntegral
import FabiusFunction.PeriodicMean
import FabiusFunction.PeriodicRegularity
import FabiusFunction.LaplacePeriodicSecondOrder
import FabiusFunction.FabiusTranslatedLegendreSeries
import FabiusFunction.FabiusLegendreLeastSquares
```

This is a deliberate top layer. It says that a user importing `FabiusFunction` should see four things at once:

1. the original analytic theory of Rvachev's function;
2. the arithmetic theory of exact dyadic values and moments;
3. the repaired and strengthened asymptotic theory;
4. later spectral, discrete, and approximation consequences.

The internal modules remain available for narrow imports, which matters for compilation latency and for understanding dependency direction.

1.4 Four kinds of source module

A useful first classification is architectural rather than mathematical.

Reusable infrastructure.
Modules such as `LegendrePolynomial.lean`, `LegendreSeriesConvergence.lean`, `SaddleAllOrders.lean`, and the `GaussianPolynomial*.lean` family prove generic analysis that is not tied to one Fabius declaration.

Fabius-specific bridges.
Modules such as `DyadicAnalytic.lean`, `GlobalExtension.lean`, `AnalyticMoments.lean`, and `EndpointLaplaceComparison.lean` connect two representations of the same object. These are usually the most informative files for understanding the library.

Paper-facing facades.
`Paper05442.lean`, `Paper06487.lean`, `PaperFabiusAsymptotic.lean`, and `PaperKFoldThueMorse.lean` expose claims in the order and vocabulary of their sources. Their proofs are mostly assembled from lower modules.

Audit and correction modules.
Files such as `DraftCounterexamples.lean`, `FabiusWikipediaObstruction.lean`, and the

comments in `PoissonSummation.lean` do something formalization is unusually good at: they state precisely which printed claim is false, identify the missing term or hypothesis, and expose a corrected theorem.

Lean engineering

The public aggregate files are intentionally thin. When debugging an elaboration or trying to reuse a proof, jump from the facade theorem to the first nontrivial bridge module in its import chain. Reading every facade from top to bottom is excellent for coverage; it is usually poor for learning how a proof works.

1.5 Naming conventions that encode intent

Several naming patterns are consistent enough to act as documentation.

Generic versus canonical.
A theorem parameterized by `(F : BoundedFabius) (hF : IsFabius F)` proves something for every implementation of the specification. A later theorem with `canonical_` or a bare `fabius_` prefix specializes it to the chosen fixed point `fabius` and its proof `fabius_spec`.

Exact-to-analytic casts.
Names ending in `_cast` bridge a rational object to a real-valued function. These theorems are the load-bearing seams between computation and analysis.

Convergence forms.
A `HasSum` theorem records the mathematically strong convergence witness; a corresponding `tsum_...` theorem is usually a convenient equality obtained with `.tsum_eq`. The same pattern appears for filters and asymptotic relations.

Integer exponents.
Names containing `_zpow` use an integer scale. This matters in Taylor reduction and small-argument asymptotics, where the relevant dyadic scale may lie on either side of 1.

Specification objects.
Predicates such as `IsFabius` and `IsOriginalFabius` separate an abstract characterization from any one construction. This enables multiple existence and uniqueness routes without redefining the downstream API.

2

The four coordinate systems of the development

A reader can understand most apparent complexity in the source by tracking which coordinate system a theorem inhabits. The library uses four primary models and several transform models built over them.

2.1 The bounded cumulative function

The core type is not simply `ℝ → ℝ`. It is a function whose output already carries the range invariant:

```
abbrev UnitInterval := Set.Icc (0 : ℝ) 1
abbrev BoundedFabius := ℝ → UnitInterval

noncomputable def fabiusReal (F : BoundedFabius) (x : ℝ) : ℝ := F x
```

Using `Set.Icc 0 1` as the codomain pays for itself repeatedly. Every value comes with nonnegativity and an upper bound, which Lean can recover as `(F x).property.1` and `(F x).property.2`. The cost is a layer of coercions whenever real algebra is performed. The wrapper `fabiusReal` provides a stable coercion boundary and gives downstream theorem statements a readable real-valued surface.

The abstract specification is a structure whose fields express the natural bounded characterization: constant tails, smoothness, midpoint symmetry, and the differential equation on the left half of the unit interval. In mathematical notation the essential content is

$$F(x) = 0 \quad (x \le 0), \qquad F(x) = 1 \quad (x \ge 1),$$

$$F(x) + F(1-x) = 1 \quad (0 \le x \le 1), \qquad F'(x) = 2F(2x) \quad (0 \le x \le \tfrac{1}{2}),$$

together with C^∞ regularity. The exact Lean fields are arranged to make endpoint proofs and composition lemmas tractable rather than to mimic one printed definition verbatim.

Proof idea

The symmetry field is deliberately local to $[0,1]$. A theorem named `symmetry_all` later combines it with the constant-tail fields to obtain a total real identity. This keeps the specification semantically sharp while providing a rewrite-friendly global theorem when needed.

2.2 Rvachev's compactly supported bump

The second model folds the bounded function about the origin to obtain the even smooth bump supported on $[-1, 1]$:

$$\mathrm{up}_F(x) = \begin{cases} F(x+1), & x \le 0, \\ F(1-x), & x > 0. \end{cases}$$

In Lean this is `rvachevUp`. The branch choice makes evenness essentially an application of the bounded symmetry law, while support follows from the constant tails. The crucial differential identity is global:

$$\mathrm{up}_F'(x) = 2\big(\mathrm{up}_F(2x+1) - \mathrm{up}_F(2x-1)\big).$$

This form drives the Fourier product, moment recurrences, smoothness bootstrap, Poisson summation, and the original paper's characterization.

2.3 The signed global extension

The bounded cumulative function is ideal near $[0, 1]$, but dyadic translation and Taylor reduction are naturally global. The library therefore defines a Thue–Morse signed superposition of translates of up:

$$\tilde{F}(x) = \sum_{n \ge 0} (-1)^{s_2(n)}\ \mathrm{up}_F(x - 2n - 1),$$

where $s_2(n)$ is the binary digit sum. This is `extendedFabius`. Although written as a `tsum`, it is locally finite: compact support means that on a bounded neighborhood only finitely many translates survive. The formalization exploits this twice:

- locally replace the infinite sum by a finite sum to prove smoothness;
- reindex even and odd terms to derive the global equation $\tilde{F}'(x) = 2\tilde{F}(2x)$.

On $[0, 1]$, `extendedFabius` agrees with `fabiusReal`; on nonpositive arguments it vanishes. Outside the unit interval it is signed rather than bounded. Confusing these two functions is the most common conceptual mistake a new reader can make.

Formalization-driven correction

The name "global Fabius function" in informal sources can hide the fact that the extension is not a cumulative distribution function on all of $\mathbb{R}$. The Lean API keeps the types separate: `BoundedFabius` has range in $[0, 1]$, while `extendedFabius` is an unrestricted real-valued function.

2.4 Exact rational coordinates

The fourth model never enters $\mathbb{R}$ until a bridge theorem asks it to. It contains:

- rational moments `moment` and `halfMoment`;
- natural division-free numerators;
- the closed rational value `fabiusDyadic` at $m/2^n$;
- executable evaluators such as `fabiusDyadicValue` and `evalFabiusDyadic`;

- exact denominator and 2-adic valuation statements.

The bridge theorem has the schematic shape

$$\text{fabiusDyadic}(n, m) \text{ cast to } \mathbb{R} = F\Big(\frac{m}{2^n}\Big),$$

after imposing the appropriate unit-interval bound, or with `extendedFabius` in the global version.

2.5 Transform and approximation coordinates

Four more representations are layered over the primary four:

Formal power series.
Exact coefficient recurrences are proved over $\mathbb{Q}[\![X]\!]$ before convergence enters the picture.

Fourier and Laplace transforms.
Compact support and smoothness permit Schwartz-space Fourier arguments; positive/negative Laplace products expose moments and endpoint asymptotics.

Probability laws.
An infinite product of uniform coordinates produces a random series whose CDF satisfies the same smoothing equation.

Finite approximants.
Atomic convolutions, step functions, uniform splines, q-binomial rows, and Legendre projections supply convergent finite models.

The library is powerful because the bridges are proved explicitly. A theorem about a rational coefficient can therefore migrate through a moment identity, become an integral formula for up, and finally control a spectral coefficient or a saddle expansion.

3 Dependency architecture

3.1 The main dependency strata

The following diagram is intentionally architectural rather than a complete import graph. An arrow points from a prerequisite to the layer that consumes it.

Three seams deserve special emphasis.

Seam 1: arithmetic to analysis

`DyadicAnalytic.lean` proves that the rational evaluator agrees with every analytic object satisfying `IsFabius`. Once this is established, exact arithmetic can be used in uniqueness, denominator, Legendre, and Taylor arguments without reproving analytic formulas each time.

Seam 2: local bump to global extension

`Differential.lean` proves the smooth refinement equation for up. `GlobalExtension.lean` combines that equation with local finiteness and Thue–Morse reindexing. Most global scaling identities depend on this seam, not directly on the contraction construction.

Seam 3: exact products to asymptotics

The asymptotic tower begins only after exact negative-Laplace product identities are established. Periodic regularization, Mellin constants, Lambert coordinates, and saddle estimates are layered afterward. This prevents asymptotic approximations from contaminating exact algebraic rewrites.

3.2 Why dependency direction matters

A tempting informal development would define the canonical function first, compute several values numerically, and then derive formulas from it. This library often reverses that order:

1. define exact rational sequences independently;
2. prove internal recurrence or power-series identities;
3. certify an executable algorithm from an abstract recurrence;
4. prove that analytic Fabius functions satisfy the same recurrence;
5. conclude equality by induction or uniqueness.

This direction is more modular. The evaluator correctness theorem does not know how the analytic function was constructed; the fixed-point construction does not know the implementation details of the evaluator; and the canonical specialization can be replaced without disturbing either side.

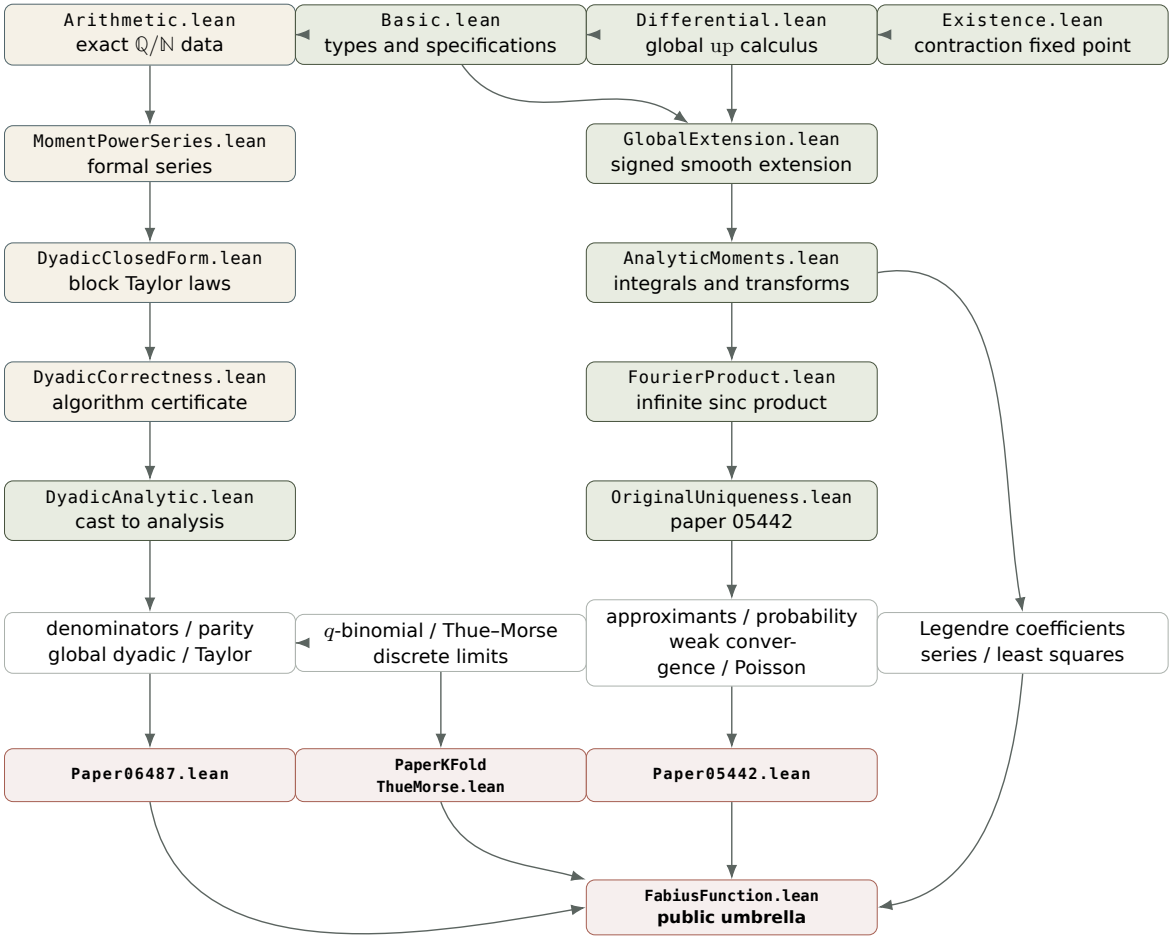

Figure 3.1: Selected dependency strata in the non-asymptotic development.

3.3 Thin facades and thick bridges

A rough rule for navigating the repository is:

$$\text{facade theorem} \longrightarrow \text{bridge theorem} \longrightarrow \text{generic infrastructure or exact core.}$$

For example, a theorem displayed in `Paper06487.lean` about a dyadic value may be one line there, a substantial cast theorem in `GlobalDyadic.lean`, a closed rational identity in `DyadicClosedForm.lean`, and a moment recurrence in `MomentPowerSeries.lean`. The one-line facade is not "where the proof is"; it is where the proof is *named in the paper's vocabulary.*

Suggested reading path

For a first source tour, read `Arithmetic.lean` → `Basic.lean` → `Differential.lean` → `Existence.lean` → `DyadicCorrectness.lean` → `DyadicClosedForm.lean` → `DyadicAnalytic.lean` → `GlobalExtension.lean`. That sequence exposes almost every recurring design pattern before the larger paper and asymptotic branches appear.

4

The exact arithmetic kernel

4.1 Why `Arithmetic.lean` comes first

`Arithmetic.lean` is mathematically elementary compared with the analytic files, but architecturally it is foundational. It defines objects over $\mathbb{N}$, $\mathbb{Z}$, and $\mathbb{Q}$ that can be reduced by the kernel and evaluated without choice, integration, limits, or real-number equality. Its imports are kept small enough that exact computation does not inherit the cost of the analytic world.

The main families are:

Binary and Thue–Morse data.
`binaryWeight` counts one-bits and `thueMorseSign` returns $(-1)^{s_2(n)}$. Theorems for even and odd indices are the reindexing engine used throughout the project.

Moment sequences.
`moment` and `halfMoment` are rational recurrences. Their division-free companions `momentNumerator` and `halfMomentNumerator` make integrality and parity induction possible.

Closed dyadic formulas.
`fabiusDyadic` is a finite exact sum for a value at a dyadic argument; `rvachevDyadic` and global signed versions adapt it to other coordinates.

Executable evaluation.
Tables of inverse-power values and a binary Taylor/Horner descent evaluate a dyadic rational much faster than directly expanding the closed sum.

Denominator data.
Products of odd factors and Mersenne-type factors are introduced in the exact type where divisibility statements belong.

4.2 Binary identities are an API, not incidental lemmas

The elementary rules

$$s_2(2n) = s_2(n), \qquad s_2(2n+1) = s_2(n) + 1$$

become sign rules

$$\mathrm{tm}(2n) = \mathrm{tm}(n), \qquad \mathrm{tm}(2n+1) = -\mathrm{tm}(n).$$

In paper proofs these are often used silently. Lean needs named theorems because they drive reindexing under finite sums and infinite sums. Later files prove the stronger block-concatenation identity

$$\mathrm{tm}(h2^k + r) = \mathrm{tm}(h)\,\mathrm{tm}(r), \qquad 0 \le r < 2^k,$$

which is the algebraic reason complete binary blocks cancel lower-degree polynomials.

Lean engineering

The project consistently proves binary facts at the natural-number level and casts their consequences afterward. Trying to reason about bit counts after casting indices to $\mathbb{Q}$, $\mathbb{R}$, or $\mathbb{C}$ would erase the structure Lean's arithmetic and bitwise lemmas can see.

4.3 Moments and division-free normalization

The moment recurrence is naturally rational. Divisibility and oddness, however, are awkward when every induction step contains normalized fractions. The code therefore maintains both the normalized sequence and a natural numerator sequence. Schematically,

$$c_n = \frac{A_n}{D_n}, \qquad d_n = \frac{B_n}{E_n},$$

where $A_n, B_n \in \mathbb{N}$ are defined recursively without division and D_n, E_n are explicit factorial/Mersenne products.

This is a recurring formalization technique:

1. state the mathematically natural recurrence in $\mathbb{Q}$;
2. discover a common denominator from the recurrence;
3. define a natural numerator by clearing that denominator;
4. prove the rational sequence equals numerator divided by denominator;
5. do parity, positivity, and divisibility on the natural sequence;
6. transport the result back to $\mathbb{Q}$.

The same split later supports `NormalizedMoments.lean`, `NormalizedEvenMoments.lean`, `HalfMomentDenominator.lean`, `DenominatorBound.lean`, and `TwoAdic.lean`.

4.4 The closed dyadic sum

The definition `fabiusDyadic` packages a finite Thue–Morse/moment formula. Its exact displayed form is less important for navigating the source than its role:

- it is total on natural scale and numerator parameters;
- it is representation-stable under $(n, a) \mapsto (n + 1, 2a)$;
- its unit-interval cast is an analytic Fabius value;
- its global cast is a value of `extendedFabius`;
- its finite nature permits denominator and congruence arguments.

The library carefully distinguishes a *closed specification* from an *algorithm*. `fabiusDyadic` is convenient for proofs because sums can be rearranged and denominators exposed. `fabiusDyadicValue` is convenient for execution because it follows the bits of the numerator. `DyadicCorrectness.lean` proves these two views coincide.

4.5 The executable evaluator

At scale n, the evaluator precomputes the values at inverse powers of two, then descends through the nonzero bits of the numerator. Conceptually, if the highest active bit moves the argument into the neighboring dyadic block, the function on that block is recovered from a finite Taylor polynomial whose coefficients are earlier inverse-power values. The implementation uses:

`fabiusInversePowTwoTable`
: a prefix table of exact rational values;

`fabiusTaylorHorner`
: a Horner evaluator for the local Taylor polynomial;

`fabiusDyadicUnitAux`
: an auxiliary bit-clearing recursion;

`fabiusDyadicUnit`
: the unit-interval algorithm;

`fabiusDyadicValue`
: the public scale/numerator evaluator;

`evalFabiusDyadic`
: a rational wrapper returning `Option` $\mathbb{Q}$, rejecting non-dyadic rationals.

The README estimates the arithmetic shape as approximately $O(n^2 + n\, s_2(a))$: the table costs a triangular amount of exact work, and each set bit triggers a Taylor/Horner block update. Bit complexity also depends on the rapidly growing rational numerators and denominators, so this estimate is best read as a count of high-level rational operations.

Proof idea

The algorithm is not certified by unfolding it against the giant closed sum. Instead the correctness layer asks for one abstract bit recurrence. The closed form proves that recurrence, and a strong-induction theorem then proves that any table satisfying it agrees with the evaluator. This is much closer to a software-verification proof than to a direct symbolic calculation.

5

The specification and the differential bootstrap

5.1 `Basic.lean`: the semantic vocabulary

`Basic.lean` imports the arithmetic kernel and enough analysis to state the central objects. It defines `BoundedFabius`, `fabiusReal`, `IsFabius`, `rvachevUp`, `extendedFabius`, complex sinc, and several transform-level definitions. The file is intentionally definition heavy: downstream modules should depend on a stable vocabulary rather than on one existence proof.

A simplified view of the specification is:

```
structure IsFabius (F : BoundedFabius) : Prop where
  zero_of_nonpos : ∀ x, x ≤ 0 → fabiusReal F x = 0
  one_of_one_le  : ∀ x, 1 ≤ x → fabiusReal F x = 1
  contDiff       : ContDiff ℝ ∞ (fabiusReal F)
  symmetry       : ∀ x ∈ Set.Icc (0 : ℝ) 1,
                     fabiusReal F x + fabiusReal F (1 - x) = 1
  hasDerivAt     : ∀ x ∈ Set.Icc (0 : ℝ) (1 / 2),
                     HasDerivAt (fabiusReal F)
                       (2 * fabiusReal F (2 * x)) x
```

The actual code chooses field formulations that interact smoothly with mathlib's coercions and endpoint lemmas, but this excerpt captures the contract. The rest of the library is mostly parametric in `F` and `hF`; the canonical fixed point appears only when a theorem is specialized for end users.

5.2 Folding to upand managing the seam at zero

The definition of `rvachevUp` is piecewise at $x = 0$. Continuity is easy from symmetry and endpoint values. Differentiability is more delicate: Lean must see that the left and right derivatives agree at the fold, and later that all higher derivatives glue.

`Differential.lean` proceeds in layers:

1. prove evenness and support bounds directly from the branches;
2. prove one-sided derivative formulas away from the fold;
3. evaluate the boundary derivatives using flatness inherited from the constant tails and the Fabius differential equation;
4. package a total `HasDerivAt` theorem;
5. bootstrap smoothness from the global derivative refinement equation.

The resulting theorem has the recognizable form:

```
theorem rvachev_hasDerivAt
    (F : BoundedFabius) (hF : IsFabius F) (x : ℝ) :
    HasDerivAt (rvachevUp F)
      (2 * (rvachevUp F (2 * x + 1) -
            rvachevUp F (2 * x - 1))) x := by
  ...
```

5.3 Why the derivative formula bootstraps all smoothness

The right side is a finite linear combination of affine compositions of `rvachevUp`. Suppose u is already C^n. Then $x \mapsto u(2x \pm 1)$ is C^n, so the displayed formula says u' is C^n. The standard equivalence between C^{n+1} regularity and a C^n derivative then gives $u \in C^{n+1}$. Starting from continuity, induction yields C^∞.

The generic lemma `contDiff_of_hasDerivAt_dyadic_refinement` isolates this pattern. It is not limited to Rvachev's exact constants; it formalizes the reusable principle "a continuously differentiable function whose derivative is a finite affine combination of copies of itself is smooth."

Lean engineering

This bootstrap avoids direct nth-derivative calculations at the piecewise seam. Proving every derivative glues by hand would duplicate endpoint bookkeeping at each order. Once the first global derivative identity is established, the generic regularity lemma lets mathlib handle the induction.

5.4 Support and positivity as analytic infrastructure

The support theorem is not cosmetic. Compact support turns upinto an integrable function, a Schwartz function after complexification, and a safe input to Poisson summation. The library records both pointwise zero theorems outside $[-1, 1]$ and set-level support inclusions. The set-level form is what integration lemmas consume.

Strict positivity on $(-1, 1)$ is proved later in `Paper06487Supplement.lean`. Combined with the derivative identity it gives strict increase on $(-1, 0)$ and strict decrease on $(0, 1)$. This is a good example of the project's layering: `Differential.lean` proves the reusable calculus statement; the paper supplement packages its elementary qualitative consequences.

6

Existence by contraction and uniqueness by dyadic density

6.1 The fixed-point space

`Existence.lean` constructs the bounded Fabius function as a Banach fixed point. The ambient space is not all continuous functions $[0,1] \to \mathbb{R}$. It is a closed subtype whose values lie in $[0,1]$ and whose members obey the midpoint symmetry. A schematic member f satisfies

$$0 \le f(x) \le 1, \qquad f(x) + f(1-x) = 1.$$

The subtype is complete because it is closed in a complete sup-norm function space. Encoding symmetry in the space, rather than proving it after the fixed point is found, makes the operator preserve an invariant that also produces the contraction constant.

6.2 The smoothing operator

The construction extends a candidate to the real line with constant tails, forms a cumulative integral, and folds/rescales it into another symmetric unit-interval function. The key numerical identity is that the total integral of a symmetric candidate on $[0,1]$ is exactly $1/2$. Indeed,

$$\int_0^1 f(t)\,dt = \frac{1}{2}\int_0^1 \big(f(t) + f(1-t)\big)\,dt = \frac{1}{2}.$$

That normalization ensures the transformed function has the correct endpoint values without a candidate-dependent denominator.

The operator's Lipschitz estimate is exactly $1/2$. On the first half of the interval the difference of cumulative integrals is bounded by the interval length times the sup norm. On the second half, symmetry rewrites the estimate back to the first. Thus

$$\|Tf - Tg\|_\infty \le \frac{1}{2}\|f - g\|_\infty.$$

Mathlib's contraction fixed-point theorem yields a unique fixed candidate in the subtype.

Proof idea

The factor $1/2$ is not an arbitrary weakened bound chosen to satisfy a library theorem. It is a structural consequence of doing all nontrivial integration on a half interval and carrying symmetry as a type invariant. This is why the choice of fixed-point space matters as much as the formula for T.

6.3 Recovering the differential specification

A fixed point of an integral operator first gives an integral equation. The file differentiates it on branch interiors and handles 0, 1/2, and 1 with one-sided or gluing arguments. The proof then invokes the differential bootstrap of `Differential.lean` to obtain C^∞ regularity. Finally the constant tails are added to produce a value of `BoundedFabius` and a proof of `IsFabius`.

The canonical declarations exposed through `PaperStatements.lean` are conceptually:

```
noncomputable def fabius : BoundedFabius := ...

theorem fabius_spec : IsFabius fabius := ...

noncomputable def globalFabius : ℝ → ℝ :=
  extendedFabius fabius
```

6.4 A second uniqueness layer

Banach's theorem already gives uniqueness inside the fixed-point subtype. The public specification `IsFabius`, however, is phrased by tails, symmetry, smoothness, and a derivative equation. The library proves that any function satisfying that specification agrees with the fixed point. The route is not "show it is another fixed point and invoke Banach uniqueness." Instead it uses the exact dyadic evaluator.

The proof has three stages.

6.4.1 Stage 1: exact equality on every dyadic point

`DyadicAnalytic.lean` proves `fabiusDyadic_cast`: an abstract Fabius function at $a/2^n$ equals the cast of `fabiusDyadic`. Hence two abstract solutions agree at every dyadic point in $[0, 1]$.

6.4.2 Stage 2: approximate an arbitrary real by floor dyadics

For $x \in [0, 1]$, let

$$x_n = \frac{\lfloor 2^n x \rfloor}{2^n}.$$

The standard bounds

$$0 \le x - x_n < 2^{-n}$$

show $x_n \to x$. Each x_n lies in the allowed dyadic range and therefore has the same value for both candidate functions.

6.4.3 Stage 3: pass to the limit by continuity

Both candidates are C^∞, hence continuous. Taking limits gives equality at x. Constant tails cover $x \notin [0, 1]$.

Lean engineering

This proof makes the arithmetic evaluator part of the semantic uniqueness story. That may look circuitous, but it buys a valuable theorem: the exact rational model is not merely a consequence of the canonical construction; it characterizes every analytic solution

on a dense set.

6.5 What the final uniqueness API gives

The file concludes with forms such as `isFabius_eq`, `existsUnique_fabius`, and an extensional equality theorem attached to the specification. Downstream code can therefore remain generic while knowing that the parameter is propositionally unique. In practice this is used in two styles:

- prove a theorem for arbitrary `F` and `hF`, then specialize to `fabius`;
- construct an object by a different method, prove `IsFabius`, and identify it with the canonical object by uniqueness.

The probability CDF representation uses the second style.

7

A second uniqueness theorem: the original compact-support problem

7.1 The original specification includes an unknown scale

The first Arias de Reyna paper starts from a compactly supported smooth function φ and a positive scale k satisfying a two-scale derivative identity. The Lean predicate `IsOriginalFabius` packages:

- C^∞ regularity;
- topological support exactly $[-1, 1]$ or the corresponding support constraints used in the source;
- strict positivity in the interior;
- normalization $\varphi(0) = 1$;
- a positive parameter k;
- the refinement equation for the derivative.

This problem is not identical to `IsFabius`. It asks Lean to recover both the function and the scale.

For fidelity to the source, positivity remains a field of the predicate. It is logically redundant: `IsOriginalFabius.mk_of_derivative_law` derives it from the other five hypotheses by applying the mean-value theorem on $[-1, 0]$, where support removes one translated term and interior positivity fixes the sign of the other.

7.2 First force the scale to be two

`OriginalCharacterization.lean` derives endpoint values, a two-term partition identity, and total integral one. Integrating or evaluating the refinement law against these normalizations forces $k = 2$. The exact route is arranged so that support removes translated terms and the remaining integral identities can be discharged without assuming the desired value in advance.

Once $k = 2$, the canonical `rvachevUp fabius` is shown to satisfy the original predicate.

7.3 Fourier uniqueness in Schwartz space

`OriginalUniqueness.lean` proves uniqueness by a route independent of the fixed-point/dyadic-density argument.

Because φ is smooth and compactly supported, its complexification is a Schwartz map. Fourier transforming the differential refinement equation gives a multiplicative recurrence.

After normalization at frequency zero, one obtains a sinc factor:

$$\widehat{\varphi}(\xi) = \operatorname{sinc}(\pi\xi)\,\widehat{\varphi}(\xi/2),$$

after matching the project's Fourier convention. Iteration gives

$$\widehat{\varphi}(\xi) = \left(\prod_{j=0}^{N-1} \operatorname{sinc}\left(\frac{\pi\xi}{2^j}\right)\right) \widehat{\varphi}(\xi/2^N).$$

As $N \to \infty$, continuity and normalization imply the last factor tends to one. Therefore every original solution has the same infinite sinc product as the canonical up. Injectivity of the Fourier transform on Schwartz space then gives equality of the functions.

Proof idea

The proof uses compact support twice: first to package the function as Schwartz, and second to ensure the Fourier transform has all the regularity and decay needed by the injectivity theorem. The infinite product is not guessed; it is the limit of a finite recurrence whose remainder frequency tends to zero.

7.4 Why both uniqueness proofs belong in the library

The two routes solve different interface problems.

Dyadic-density uniqueness
: identifies all bounded cumulative solutions of `IsFabius`. It emphasizes exact arithmetic and continuity.

Fourier uniqueness
: identifies all compactly supported smooth original solutions and determines the unknown scale. It emphasizes transform theory and support.

Neither is redundant. The first certifies exact computation as a complete dense fingerprint; the second mirrors the original mathematical characterization and builds infrastructure later reused for the Fourier product and Poisson summation.

8

Formal power series and analytic moments

8.1 The formal-series layer is deliberately pre-analytic

`MomentPowerSeries.lean` proves the moment recurrences as identities in formal power series. This removes all radius-of-convergence and interchange questions. A representative identity rescales the moment series and factors it through a hyperbolic-sine quotient:

$$M(4X) = M(X)\,\frac{\sinh\sqrt{X}}{\sqrt{X}},$$

with the precise indexing and normalization encoded by the project's `momentPS` and auxiliary series. The point is structural: coefficient extraction from a formal product exactly reproduces the finite recurrence.

The half-moment series is then characterized by another functional equation. The file constructs a candidate, proves the functional equation, and proves uniqueness coefficient by coefficient with strong induction. Even/odd coefficient filtering yields the key relations among `moment`, `halfMoment`, and inverse-power Fabius values.

Lean engineering

When an identity is fundamentally coefficient algebra, the code stays in `PowerSeries ℚ`. Analytic convergence appears only in modules whose conclusions are genuinely about functions, integrals, or transforms. This separation makes recurrence proofs faster, more exact, and much easier to reuse in denominator arguments.

8.2 From the bump to integral moments

`AnalyticMoments.lean` begins on the analytic side. Compact support and evenness permit all whole-line integrals to be localized. The normalization

$$\int_{\mathbb{R}} \mathrm{up}_F(x)\,dx = 1$$

is derived from the differential/refinement equation rather than installed as an axiom.

Integration by parts then converts the derivative equation into moment recurrences. The boundary terms vanish because of compact support and flatness. For even powers, symmetry kills odd contributions and leaves the same rational recurrence used by `moment`. Uniqueness of the recurrence identifies the integral with the exact sequence:

$$\int_{-1}^{1} x^{2n}\,\mathrm{up}_F(x)\,dx = c_n.$$

A half-interval version similarly identifies `halfMoment`. The additive theorem `halfMoment_eq_integral_formula_all` includes the normalized zeroth term, where both exact and analytic definitions equal one, without changing the source-faithful positive-index theorem.

8.3 The inverse-power identity

One of the most important bridges in the library relates the value at 2^{-n} to a half moment. In normalized schematic form,

$$F(2^{-n}) = \frac{d_n}{n!\, 2^{\binom{n}{2}}},$$

where d_n is the project's `halfMoment n` up to the exact indexing used in the declaration. This theorem appears in multiple useful forms:

- an exact rational identity in `MomentPowerSeries.lean`;
- a real analytic formula for arbitrary `F` and `hF`;
- a table-entry theorem used by the executable evaluator;
- a denominator/valuation input;
- a coefficient source for Legendre expansions.

The declaration `halfMomentFabiusValue` packages the rational normalization, while `fabiusAtInverseTwoPow_eq_halfMoment` and cast variants connect it to the function.

8.4 Generating functions, Laplace transforms, and Fourier transforms

Once moments are integrals, exponential series can be interchanged with the compactly supported integral. The code obtains generating-function and Laplace identities by dominated convergence or standard integral-series lemmas. Complexification yields the Fourier transform at complex arguments, which is later identified with the infinite sinc product.

A useful navigation rule is:

- `MomentPowerSeries.lean`: exact coefficient algebra;
- `AnalyticMoments.lean`: integral identification and transform series;
- `LaplaceTransform.lean`, `LaplaceMoments.lean`: transform-specific packaged theorems;
- `FourierAnalytic.lean`, `FourierProduct.lean`: entire/Fourier consequences and product evaluation.

8.5 Normalized moments and denominator chains

`NormalizedMoments.lean` and `NormalizedEvenMoments.lean` make common denominators explicit. A typical proof proceeds by induction on the recurrence, using the fact that earlier denominator products divide later ones. The normalizing product is chosen to be multiplicative and monotone under index, which lets Lean combine divisibility witnesses mechanically.

`HalfMomentDenominator.lean` sharpens the half-moment normalization. `TwoAdic.lean` then proves parity of the normalized numerator, often by showing that all but one term in

a recurrence are even and identifying the surviving Pascal-row contribution. The resulting 2-adic valuations are statements about exact rationals, not numerical experiments.

9

Certified exact evaluation at dyadic rationals

9.1 Four files, four proof obligations

The evaluator story is split across four central modules.

Module	Responsibility
`Arithmetic.lean`	Define the closed sum, inverse-power table, Horner routine, bit recursion, and public exact evaluator.
`DyadicCorrectness.lean`	Prove representation stability and show that an abstract bit recurrence is sufficient for implementation correctness.
`DyadicClosedForm.lean`	Derive the needed bit/block recurrence from Thue–Morse cancellation and polynomial/Taylor algebra.
`DyadicAnalytic.lean`	Prove that every analytic Fabius function obeys the same recurrence and identify its values with the rational evaluator.

This decomposition prevents a cyclic proof. The algorithm is specified before analysis; the closed rational formula is proved correct without using the canonical function; the analytic function is linked only after both exact pieces are stable.

9.2 Representation refinement

A dyadic number has infinitely many scale/numerator representations:

$$\frac{a}{2^n} = \frac{2a}{2^{n+1}} = \frac{4a}{2^{n+2}} = \cdots.$$

Before a public rational evaluator can be trusted, the code proves that its value is invariant under this refinement. This theorem is separate from function correctness. It says that the implementation computes a well-defined function of the dyadic rational, rather than a function of a chosen encoding.

The wrapper `evalFabiusDyadic` analyzes a rational denominator and returns `none` precisely when it is not a power of two after reduction. Its correctness API includes both directions: accepted inputs have a certified exact value, and rejected inputs are genuinely non-dyadic.

9.3 Prefix stability of the inverse-power table

The inverse-power table is built incrementally. Later scales append entries but must not alter earlier ones. `DyadicCorrectness.lean` proves:

- the exact table length;
- an append recurrence for the next inverse-power value;
- lookup stability under extension;
- Horner evaluation invariants for prefixes;
- compatibility between auxiliary and public evaluators.

These are ordinary program invariants, and their separation from the mathematical recurrence is exemplary. A change in table representation would mostly affect this file, not the analytic bridge.

9.4 The abstract bit recurrence

The central interface is a predicate of the form `FabiusDyadicHasBitRecurrence`. Informally it says: when the highest nonzero bit of the numerator is removed, the value in the corresponding binary block is obtained by a finite Taylor expression using the inverse-power table. The exact predicate carries the necessary range and scale conditions.

A strong-induction theorem then has the shape:

```
theorem fabiusDyadicUnit_eq_of_hasBitRecurrence
    (table : List ℚ)
    (hrec : FabiusDyadicHasBitRecurrence table) :
    ∀ n a, ... →
      fabiusDyadicUnit table n a = fabiusDyadic n a := by
  intro n a
  induction a using Nat.strong_induction_on with
  | h a ih =>
      -- clear the highest bit; recursive arguments are smaller
      ...
```

The proof follows the executable control flow. The recursion is justified not by decreasing scale but by decreasing numerator after clearing a set bit. That is why strong induction on a, rather than ordinary induction on n, is the natural principle.

9.5 Thue–Morse cancellation creates Taylor blocks

`DyadicClosedForm.lean` supplies the mathematical recurrence. Its first major ingredient is the Prouhet cancellation identity

$$\sum_{r=0}^{2^b-1} (-1)^{s_2(r)} r^d = 0 \qquad (d < b),$$

and the normalized nonzero value at $d = b$. The file proves affine and translated variants, because the dyadic closed form contains powers of shifted indices rather than bare r^d.

The second ingredient is a polynomial kernel whose Hasse derivatives encode local Taylor coefficients. A refinement identity is established by showing that the derivatives of two polynomials agree and their constant terms agree. This is often more robust in Lean than expanding both sides into enormous binomial sums.

Complete binary blocks annihilate all powers below the critical degree. What survives is exactly the coefficient needed for the next inverse-power table entry. Partial blocks produce a Taylor polynomial. These facts are packaged as a block Taylor property, then converted to the abstract bit recurrence.

Proof idea

The highest-bit step works because binary concatenation factors the Thue–Morse sign and because a complete 2^b block acts like a finite-difference operator of order b. Lower-degree Taylor terms cancel; the first nonzero term has a closed normalization. The evaluator is therefore a compressed traversal of the same cancellation already present in the closed formula.

9.6 The analytic recurrence

`DyadicAnalytic.lean` independently proves a Taylor-block recurrence for any `F` satisfying `IsFabius`. The key analytic input is the iterated derivative scaling law, eventually available in global form:

$$\tilde{F}^{(k)}(x) = 2^{\binom{k+1}{2}} \tilde{F}(2^k x).$$

At dyadic base points these derivatives are inverse-power values already in the table. A finite Taylor formula is exact on the relevant block because the translated remainder cancels by the signed scale-translation identity; in the local unit-interval proof, the same effect is obtained through the refinement recurrence.

The file proves the first scale separately, then inducts through higher scales. It identifies the analytic Taylor sum with the cast of `fabiusTaylorHorner`, proves the bit recurrence, and applies the same strong-induction framework used by the exact closed form. The final theorem is `fabiusDyadic_cast`.

9.7 Why this is a particularly strong formalization result

The result is more than "the first few dyadic values are rational." It gives:

- a total exact algorithm for every dyadic rational in the domain;
- proof that different encodings give the same answer;
- proof that the algorithm equals a closed finite formula;
- proof that the exact result equals every analytic solution;
- exact denominator and valuation consequences downstream.

The code thus joins executable mathematics, algebraic combinatorics, and functional analysis at a single certified interface.

10

The signed global extension and dyadic translation calculus

10.1 Local finiteness hidden inside a `tsum`

The definition

$$\tilde{F}(x) = \sum_{n \geq 0} \mathrm{tm}(n)\, \mathrm{up}_F(x - 2n - 1)$$

uses a topological sum because that is the natural public expression. Yet the support of upmeans the summand can be nonzero only when

$$-1 < x - 2n - 1 < 1,$$

so at each fixed x at most one or two neighboring indices matter, depending on endpoint convention. On a sufficiently small neighborhood the same finite set of indices works uniformly.

`GlobalExtension.lean` converts this observation into local finite-sum equalities. The proof pattern is:

1. choose integer bounds from the point x and a small radius;
2. prove all indices outside a finite range evaluate outside the support;
3. rewrite the `tsum` as a finite `Finset.sum`;
4. apply finite-sum continuity or smoothness;
5. transfer the local result back to `extendedFabius`.

This is more robust than trying to prove uniform summability of all derivatives for a series that is actually locally finite.

10.2 Agreement with the bounded coordinate

For $x \in [0, 1]$, only the first translated bump contributes, and its branch is exactly the bounded cumulative function. The file proves `extendedFabius_eq_fabiusReal` on the closed unit interval. For $x \leq 0$, all translates lie outside support and `extendedFabius_eq_zero_of_nonpos` provides flatness at the origin.

A block-local theorem identifies the unique surviving translate on intervals of length two. This is the basic simplifier used in global dyadic and translation proofs: instead of unfolding a global infinite sum, choose the block containing x and reduce to one upvalue with a Thue–Morse sign.

10.3 Reindexing the derivative

Differentiate termwise after replacing the series locally by a finite sum. Each bump derivative contributes

$$2\,\mathrm{up}_F(2x-4n-1)-2\,\mathrm{up}_F(2x-4n-3).$$

The two families correspond to even and odd indices in the extension evaluated at $2x$. The identities

$$\mathrm{tm}(2n)=\mathrm{tm}(n),\qquad \mathrm{tm}(2n+1)=-\mathrm{tm}(n)$$

make the signs align, yielding

$$\tilde{F}'(x)=2\tilde{F}(2x).$$

In Lean, the hard part is not differentiation but exact reindexing of two locally finite sums. The file supplies explicit even/odd bijections and uses support to justify the chosen finite ranges.

10.4 The closed formula for all iterated derivatives

Repeated differentiation gives the triangular power of two

$$\tilde{F}^{(k)}(x)=2^{1+2+\cdots+k}\tilde{F}(2^kx)=2^{\binom{k+1}{2}}\tilde{F}(2^kx).$$

The formal theorem is named `iteratedDeriv_extendedFabius`. The induction must reconcile three pieces of syntax:

- `iteratedDeriv (k+1)` as a derivative of `iteratedDeriv k`;
- the chain-rule factor 2^k from differentiating $x\mapsto 2^kx$;
- the combinatorial identity $\binom{k+2}{2}=\binom{k+1}{2}+k+1$.

This theorem is one of the most reused declarations in the subtree. It feeds Taylor reduction, inverse-power identities, flatness, translated splines, and endpoint asymptotics.

10.5 Power-of-two translation and Thue–Morse sign change

`ScaleTranslation.lean` proves that on the initial block,

$$\tilde{F}(x+2^r)=-\tilde{F}(x),\qquad 0\le x\le 2^r,$$

for $r\ge 1$. The proof truncates both global sums to finite ranges. The translated range is the next binary block, and the binary weight identity

$$s_2(2^{r-1}+m)=s_2(m)+1\qquad (0\le m<2^{r-1})$$

flips every sign. The support arguments ensure that no other translates survive.

Combining translation with the iterated derivative formula gives a sign change for sufficiently high derivatives translated by a dyadic scale, even when the scale is an integer rather than a natural. This is the technical form needed by Taylor remainder cancellation.

10.6 Exact cancellation of neighboring Taylor remainders

Suppose x lies in the dyadic annulus

$$2^s \le x < 2^{s+1}.$$

Write $x = 2^s + y$ with $0 \le y < 2^s$. The integral remainder in the Taylor expansion about 2^s is transformed by $t \mapsto t + 2^s$. The derivative translation theorem changes its sign, and the polynomial kernel $(x-t)^n$ becomes $(y-t)^n$. Therefore the remainder about 2^s is the negative of the remainder about zero at y.

`TaylorReduction.lean` combines the two Taylor formulas. Since every iterated derivative of `extendedFabius` vanishes at zero, the expansion at zero consists only of the remainder. The expansion at the inverse-power base has coefficients expressible through half moments. Adding the equations cancels the remainders and yields a *finite exact* reduction formula.

```
theorem extendedFabius_reduction
    (F : BoundedFabius) (hF : IsFabius F)
    (x : ℝ) (n : ℤ)
    (hx : 0 < x)
    (hlo : (2 : ℝ) ^ (-n) ≤ x)
    (hhi : x < (2 : ℝ) ^ (-n + 1)) :
    let y := x - (2 : ℝ) ^ (-n)
    extendedFabius F x = -extendedFabius F y +
      if 0 ≤ n then fabiusReductionSum n.toNat y else 0 := by
  ...
```

For nonnegative n, the finite sum is a moment-normalized Taylor polynomial. For negative n, the statement reduces directly to the power-of-two sign translation law and the correction term is zero.

Proof idea

The formula is exact despite using Taylor's theorem because the remainder is not estimated; it is paired with an adjacent remainder and canceled by a Thue–Morse translation identity. This "Taylor expansion plus exact remainder symmetry" is a signature technique of the arithmetic paper branch.

11

The first paper facade: compact support, probability, and approximation

11.1 What `Paper05442.lean` exposes

`Paper05442.lean` is the facade for Juan Arias de Reyna's *An infinitely differentiable function with compact support: definition and properties*. It imports:

- `OriginalUniqueness.lean` for Theorem 1;
- `WeakConvergence.lean` for the early probability measures;
- `StepApproximationLimit.lean` for the pointwise finite-step limit;
- `ProbabilityRepresentation.lean` for the infinite product model;
- `PoissonSummation.lean` for partitions of unity;
- `OriginalPaperSupplement.lean` for prose-level identities and corrected forms not represented as numbered theorem environments.

The facade is best read beside the paper. The lower modules are best read when trying to understand or extend a proof.

11.2 Finite convolution and atomic approximants

`EarlyApproximants.lean` builds finite probability measures from centered Bernoulli or uniform ingredients at dyadically shrinking scales. Their characteristic functions are finite products of cosine or sinc factors, the finite analogues of the Fourier product of up.

The same coefficients are repackaged as step functions. Half-weighted endpoint indicators are used so that convolution identities remain correct at jump points. This detail matters: ordinary closed or open interval indicators give the right function almost everywhere but fail at the exact endpoints where the paper states pointwise identities.

Formalization-driven correction

The formalization records two source-level repairs. The equation corresponding to the finite convolution must use the finite convolution object actually defined at that stage, and the step approximation theorem needs half weights at cell endpoints. These are not measure-theoretic quibbles: without them the literal pointwise formula is false on a finite set.

11.3 From interval averages to pointwise convergence

`StepApproximationLimit.lean` proves convergence without differentiating the step functions. First it establishes coefficient unimodality, which gives monotonicity on one side of the center and antitonicity on the other. Then it proves convergence of integrals over intervals. A generic squeezing theorem says, roughly:

> If monotone approximants have convergent integrals over every sufficiently small interval and the limiting density is continuous, then the pointwise values converge to the density.

For a monotone function, the value at a point is trapped between the left and right interval averages. As the interval shrinks, continuity makes both limiting averages approach the same value. The file packages this argument so that the final theorem `stepApproximant_tendsto_rvachevUp` is a clean specialization.

Lean engineering

The generic interval-average lemma is stronger and cleaner than a proof tied to the explicit coefficients. All coefficient combinatorics is discharged before the limiting argument begins. This is a common pattern in the project: make the final analytic convergence theorem depend on a compact abstract interface.

11.4 Weak convergence through characteristic functions

`WeakConvergence.lean` treats the corresponding probability measures. The finite characteristic function is a product over the first n dyadic scales. The target characteristic function is the infinite sinc product associated with up.

The proof separates a finite head and an infinite tail. The head converges by literal stabilization. For the tail, a local estimate

$$\operatorname{sinc}(z) = 1 + O(z^2)$$

and the geometric sum of squared dyadic scales show the omitted product tends to one. In the finite approximants an additional exponent n appears; the relevant elementary limit is of the form $n2^{-n} \to 0$.

Once pointwise convergence of characteristic functions and continuity at zero are established, mathlib's Lévy continuity theorem gives weak convergence of probability measures. The file also exports the bounded-continuous test function formulation, which is often more useful downstream than the abstract measure topology statement.

11.5 The infinite product probability representation

`ProbabilityRepresentation.lean` gives a different construction. Its sample space is

$$\Omega = [0,1]^{\mathbb{N}}$$

with product Lebesgue probability measure, represented in Lean as functions `ℕ → Set.Icc`

`(0 : ℝ) 1`. Define

$$X(\omega) = \sum_{n=0}^{\infty} \frac{\omega_n}{2^{n+1}}.$$

Uniform convergence against the geometric majorant gives continuity and measurability. Pointwise bounds show $0 \le X \le 1$.

The head/tail decomposition is exact:

$$X(\omega) = \frac{\omega_0 + X(\operatorname{tail}\omega)}{2}.$$

The code proves that the first coordinate and the tail process are independent, that deleting the first coordinate preserves the product measure, and hence that the law μ of X is self-similar:

$$\mu = (\lambda_{[0,1]} \times \mu) \circ \left((u,v) \mapsto \frac{u+v}{2}\right)^{-1}.$$

11.6 The CDF satisfies the Fabius smoothing equation

Let $H(y) = \mu((-\infty, y])$. Conditioning on the first uniform coordinate gives

$$H(y) = \int_0^1 H(2y-u)\,du.$$

The Lean proof is explicit: rewrite the CDF as a measure of a measurable set, use the map equation for μ, apply product integration/Fubini to the indicator, and identify each fiber measure with $H(2y-u)$.

Coordinatewise reflection $\omega_n \mapsto 1-\omega_n$ preserves product measure and sends X to $1-X$. Therefore $H(y) + H(1-y) = 1$ on $[0,1]$. Differentiating the smoothing equation or using the cumulative operator shows that H satisfies `IsFabius`. The uniqueness theorem from `Existence.lean` then identifies H with `fabiusReal fabius`.

Proof idea

Probability is used as an alternative construction, not as an assumption about the canonical fixed point. The law is built first; self-similarity yields the same analytic specification; uniqueness supplies identification. This keeps the probabilistic representation honest and modular.

11.7 Fourier product and Poisson summation

`FourierProduct.lean` defines complex sinc as the divided slope of sine at zero, making it an entire function without a special-case singularity in later analytic theorems. Local estimates prove convergence of

$$\prod_{n\ge 0} \operatorname{sinc}\left(\frac{\pi z}{2^n}\right)$$

uniformly on compact subsets. The Fourier refinement recurrence produces the finite product; the residual transform at $z/2^N$ tends to its value at zero, which is one. Thus the Fourier transform of upis exactly the infinite product.

For Poisson summation, `PoissonSummation.lean` packages a positively rescaled upas a Schwartz map and computes its Fourier transform under scaling. The same construction

exposes `rvachevFourier_real_iteratedDeriv_rapidDecay`: for every pair of natural indices, an arbitrary polynomial frequency weight times the corresponding real-axis derivative is uniformly bounded. Its specialization `rvachevFourier_real_rapidDecay` is the rapid decay of the transform itself. Mathlib's Schwartz Poisson theorem then gives

$$\sum_{k\in\mathbb{Z}} \mathrm{up}_F(t+uk) = \frac{1}{u}\sum_{k\in\mathbb{Z}} \widehat{\mathrm{up}_F}(k/u)e^{2\pi ikt/u} \qquad (u>0).$$

The sinc product vanishes at every nonzero integer because its first factor already vanishes. Taking $u = 1/n$ leaves only the zero Fourier mode and yields

$$\sum_{k\in\mathbb{Z}} \mathrm{up}_F\left(t+\frac{k}{n}\right) = n.$$

In particular, integer translates form a partition of unity.

Formalization-driven correction

The printed exponential in equation (25) of the source omits its dependence on t. The formal theorem includes $e^{2\pi ikt/u}$, as forced both by Poisson summation and by the later specialization. The module also records a correction to equation (32): after multiplying the preceding identity by the scale parameter, the displayed factor $1/a$ must disappear.

12 The arithmetic paper facade

12.1 Numbered statements versus prose-level support

`Paper06487.lean` exposes the numbered results of *Arithmetic of the Fabius function*. Most theorem bodies are short applications of `PaperStatements.lean` and `Paper06487Supplement.lean`. The latter collects claims used in prose or in proofs but not set off as numbered theorem environments.

This split is useful for maintenance:

- `PaperStatements.lean` is the stable numbered API;
- `Paper06487Supplement.lean` records exact auxiliary assertions, corrections, and expanded displayed formulas;
- lower modules remain organized by mathematical mechanism rather than paper order.

12.2 Qualitative consequences

The supplement proves nonnegativity of all moments from their exact recurrence, strict positivity of upon the interior of its support, and signs of its derivative on the two halves. It also exposes the elementary identity

$$\mathrm{up}_F(t) + \mathrm{up}_F(1-t) = 1 \qquad (0 \le t \le 1),$$

which follows by reducing each branch to bounded Fabius symmetry.

Every derivative of `extendedFabius` vanishes at zero. More generally, for positive integer scale, all derivatives vanish at 2^s. The proof uses the iterated derivative formula and the power-of-two sign translation at the origin. These flatness results justify exact Taylor manipulations that would otherwise leave endpoint terms.

12.3 Global dyadic formulas

`GlobalDyadic.lean` transports the unit-interval exact formula to the signed extension. A real dyadic point is reduced to a length-two block, the unique surviving translate is identified, and the Thue–Morse sign of the block is factored out. This is the global analogue of representation refinement: different block/scale decompositions must give the same rational value.

The supplement presents the "shifted Rvachev" formula in both compact and fully expanded forms. For an integer a in the allowed range,

$$\mathrm{up}_F\left(\frac{a-2^n}{2^n}\right) = \mathrm{fabiusDyadic}(n,a),$$

then unfolds the right side into the paper's finite double sum. Keeping both forms is valuable: the compact theorem rewrites well; the expanded theorem matches a printed formula exactly.

12.4 Denominator bounds

`DenominatorBound.lean` starts from the exact finite dyadic formula. It chooses an explicit common denominator, multiplies the sum by it, and proves the result is a natural number. From

$$D \cdot q \in \mathbb{Z}$$

for a reduced rational q, a generic rational lemma concludes that the reduced denominator of q divides D. Taking a least common multiple over a finite dyadic grid gives the paper's denominator bound.

The proof is organized to avoid direct manipulation of `Rat.den`: all complicated algebra is done before reduction, while a small generic theorem handles the final divisibility consequence of reduced form.

12.5 Odd normalizations and 2-adic valuations

`TwoAdic.lean` proves that certain normalized moment numerators are odd. The main recurrence argument separates terms by parity and uses exact facts about binomial coefficients. Once a rational is written as an odd integer divided by an explicit power of two times odd factors, `padicValRat 2` simplifies to the visible exponent.

The paper supplement records a corrected proof-internal factor: in a theorem-20 normalization, the natural quantity is $2d_kN_n$, not d_kN_n. The Lean theorem states the corrected product and then strengthens it to an odd natural number. This correction propagates cleanly because normalization facts are isolated from the final numbered theorem.

Formalization-driven correction

The arithmetic facade does not silently "make the proof go through." Where a printed sentence omits a factor or a lemma needs an extra range hypothesis, the source comment names the discrepancy and the public declaration states the repair. This turns the formalization into an auditable critical edition of the mathematics.

12.6 Taylor reduction as the computational heart of the paper

Proposition 10's recursive formula is assembled from `ScaleTranslation.lean`, `TaylorReduction.lean`, inverse-power values, and flatness. It reduces an argument in one dyadic annulus to a smaller translated argument plus an explicit polynomial in the displacement. Iterating the reduction follows the binary expansion of the argument, which explains the close relationship between the paper formula and the exact bit evaluator.

The formalization has two advantages over a purely handwritten recursion:

- integer scales make the negative-index branch explicit rather than tacitly assuming the argument is below one;
- every use of Taylor's theorem carries its domain, differentiability, and interval-integrability hypotheses, so endpoint flatness cannot be accidentally omitted.

13

Thue–Morse generating identities and q-binomial formulas

13.1 The combinatorial sublibrary

The exact formulas beyond the two papers depend on a substantial Thue–Morse sublibrary:

`ThueMorsePrefix.lean`
finite prefixes, parity splitting, block sums, and iterated prefix convolution;

`ThueMorseGenerating.lean`
polynomial and formal generating identities;

`ThueMorseExponential.lean`
finite exponential sums and their binary product factorization;

`ThueMorseApproximation.lean`
corrected approximation statements and error bounds;

`ThueMorseBinomialLog.lean`
the zero–one sequence, binomial parity, and exact base-two logarithm formulas;

`HalfQBinomial.lean`
Gaussian binomial coefficients at $q = 1/2$ and finite Pochhammer products.

These modules are not merely support code for one formula. They make the binary cancellation mechanisms reusable in discrete limits, splines, and asymptotic comparison results.

13.2 Finite exponential factorization

For a binary prefix of length 2^n, the signed exponential sum factors:

$$\sum_{r=0}^{2^n-1} \mathrm{tm}(r) e^{rz} = \prod_{j=0}^{n-1} (1 - e^{2^j z}).$$

The proof is an induction that splits the next block into even and odd halves. The sign rules turn the odd half into a negative shifted copy of the even half, producing the next factor. Differentiating or extracting coefficients gives the Prouhet power-sum cancellation used in the dyadic and spline branches.

13.3 q-weights as coefficient extractors

`FabiusQBinomialFormula.lean` is a large algebraic bridge. Its central idea is to choose a finite family of q-binomial weights whose moments satisfy

$$\sum_j w_j j^d = 0 \quad (d < n), \qquad \sum_j w_j j^n = \text{explicit nonzero normalization.}$$

Thus the weighted sum acts as an exact finite coefficient extractor of order n. If a polynomial is translated, the binomial theorem introduces only lower powers; the weights annihilate them. This proves a stronger common translation invariance than the final displayed formula strictly needs.

The code first establishes formal recurrence series and refinement factors, then isolates the weighted coefficient, then inserts the Thue–Morse exponential factorization. Only at the end are all subtractions and powers specialized to $\mathbb{Q}$ and reorganized into the paper-style sum.

Proof idea

The q-binomial sum is not verified by brute-force expansion. It is designed as a dual basis functional: lower-degree components vanish by construction, and the target coefficient has a closed normalization. Once that abstraction is proved, translation and arbitrary-numerator formulas become short algebraic consequences.

13.4 Inverse powers first, arbitrary numerators second

The first exact formula targets $F(2^{-n})$, where the inverse-power moment identity supplies a simple anchor. The arbitrary dyadic numerator is then obtained by decomposing the relevant Thue–Morse prefix into binary blocks and using the translation-invariant coefficient extractor on each block.

The principal declarations include variants of

- `fabiusDyadic_eq_qBinomialThueMorseDyadicTranslatedFormula`;
- half-shift and unshifted finite-sum forms;
- real casts for bounded Fabius values;
- global signed casts for `extendedFabius`;
- scalar-generalized formulas for real and complex targets.

The proliferation of variants is purposeful. One is good for exact computation, one for matching a symbolic formula, one for translation, and one for feeding a limiting argument.

13.5 Raw, scalar, and Taylor layers

The surrounding files reveal the proof decomposition:

`FabiusRawQBinomialFormula.lean`
preserves the unrearranged finite expression closest to direct expansion.

`FabiusRawQBinomialScalar.lean`
generalizes the scalar field while keeping the raw combinatorics.

FabiusQBinomialScalarFormula.lean
exports the simplified formula in a general scalar setting.

FabiusQBinomialTaylor.lean
connects the weighted sums to finite Taylor expansions.

FabiusDyadicQBinomialScalar.lean
specializes the scalar result to exact dyadic coordinates.

FabiusGlobalQBinomialSeries.lean
packages global signed-series consequences.

FabiusInverseDyadicClosedForm.lean
collects exact inverse-dyadic closed forms in final notation.

13.6 Audit of the local K-fold Thue–Morse draft

`PaperKFoldThueMorse.lean` is explicitly a claim-level audit. The associated TeX draft contains numbered equations and prose claims rather than theorem environments, so the facade imports both proofs and counterexamples. It certifies finite Thue–Morse identities, iterated-prefix convolution, exact zero runs, generating series, lower-Lambert branch facts, corrected pointwise approximation, and coarse Fabius decay estimates.

It also exposes machine-checked refutations of the literal normalization in the draft's equation (1), local and global error claims based on that normalization, the claimed maximum in equation (7), and an equation-(10) proxy missing a linear term. The lower Lambert branch is formalized after making the domain and integer-to-real transition explicit, but it does not repair the earlier false claims.

Lean engineering

A counterexample module is part of the trusted mathematical product. It keeps known false intermediate claims from being rediscovered and accidentally used in future asymptotic work. The facade's comments distinguish "proved after repair" from "refuted literally," which is much more informative than simply omitting failed lemmas.

14

Discrete limits, Toeplitz rows, and uniform splines

14.1 Why there is a separate discrete-limit branch

The q-binomial identities yield finite expressions that resemble an approximation scheme: an outer finite row of weights combines inner Thue–Morse polynomial prefixes at increasing scales. Turning that symbolic formula into a limit requires three logically separate results:

1. the inner prefix, after normalization, converges to the desired Fabius or spline quantity;
2. the outer q-binomial weights sum to one and have uniformly bounded total variation;
3. every inner scale sampled by row n tends to infinity uniformly over the finite row.

The modules `FabiusDiscreteLimitIntegration.lean`, `FabiusDiscreteLimitComplexShift.lean`, and `FabiusDiscreteLimitToeplitz.lean` isolate these obligations.

14.2 Safe encoding of a floor-defined finite prefix

The inner summation endpoint is written in informal formulas as an inclusive floor such as

$$\left\lfloor 2^p x - \frac{1}{2} \right\rfloor .$$

A Lean `Finset.range` wants a natural *length*, not a possibly negative inclusive endpoint. The definition

```
def fabiusDiscreteLimitRangeLength (x : ℝ) (p : ℕ) : ℕ :=
  ⌊(2 : ℝ) ^ p * x + 1 / 2⌋₊
```

is the robust translation. For $x \geq 0$ the file proves that its integer cast is the informal floor plus one. The empty case is automatic: when the printed upper endpoint is -1, the natural floor length is zero. A separate theorem characterizes exactly when the range is empty.

Lean engineering

This small definition prevents a large class of off-by-one and negative-bound proofs. It also centralizes the convention "nearest integer, with half integers rounded upward," so later formulas do not each re-prove floor arithmetic.

14.3 The outer q-binomial row

After the substitution $j = n - k$, the outer coefficient is packaged as `discreteLimitWeight`. The file proves three Toeplitz conditions.

14.3.1 Row sums equal one

A finite q-binomial theorem at $q = 1/2$ evaluates the row sum. The denominator is the finite Pochhammer product `halfQPochhammer n`, which is nonzero by positivity. Algebraic normalization converts the theorem's standard exponent to the triangular exponent in the weight.

14.3.2 Uniform total variation

The absolute row sum is bounded by an explicit constant (the file proves the convenient bound 16). This is obtained by evaluating a related positive q-binomial sum at $-1/2$, bounding a product against the reciprocal of `halfQPochhammer`, and proving the elementary lower bound

$$\mathrm{halfQPochhammer}(n) \geq \frac{1}{4}.$$

The constant is not optimized; its role is uniform boundedness.

14.3.3 Indices escape to infinity

Every sampled inner scale has the form $n + j$ or a comparable affine expression, so the minimum over $0 \leq j \leq n$ tends to infinity. The code packages this as a quantified tail condition rather than relying on an informal "clearly all indices are large."

14.4 The finite-row Toeplitz theorem

The generic theorem `tendsto_weighted_rows_of_tendsto` is a finite-row version of Toeplitz summability. Let $H_m \to L$. If

$$\sum_j w_{n,j} = 1, \qquad \sum_j \|w_{n,j}\| \leq C,$$

and every index $i(n, j)$ in row n eventually lies beyond any prescribed threshold, then

$$\sum_j w_{n,j} H_{i(n,j)} \longrightarrow L.$$

The proof is a clean norm estimate. Rewrite the error using the row-sum identity:

$$\sum_j w_{n,j} H_{i(n,j)} - L = \sum_j w_{n,j} (H_{i(n,j)} - L).$$

Choose a tail where every pointwise error is below $\varepsilon/(C + 1)$, apply the triangle inequality, and use the variation bound. The theorem is polymorphic over an `RCLike` scalar, so the same argument serves real, complex, rational-shift, and Gaussian-rational-shift variants.

14.5 Complex shifts and why polynomial translation is harmless

The inner Thue–Morse polynomial allows a scalar translation q. Lower-degree terms introduced by replacing r with $r+q$ cancel over complete binary blocks. The surviving degree is controlled by the same Prouhet normalization used in the exact formulas. The complex-shift module proves this uniformly in $q \in \mathbb{C}$, then obtains real and rational forms by specialization.

This is a recurring advantage of proving the algebra over an abstract `RCLike` field: the difficult finite cancellation is done once, while analytic limits choose the scalar needed by the application.

14.6 The half-cell-centered uniform spline

`FabiusUniformSpline.lean` defines

$$S_p(x) = \frac{(-1)^p}{2^{\binom{p}{2}} p!} \sum_{r<\lfloor 2^p x+1/2 \rfloor} \mathrm{tm}(r) \left(r - 2^p x + \frac{1}{2}\right)^p.$$

The half-cell shift matches the safe range convention. The module proves several structural facts directly from finite sums:

- the zeroth spline is a bounded Thue–Morse prefix sum;
- affine power sums of degree below the block size vanish;
- the critical degree has normalization $(-1)^p 2^{\binom{p}{2}} p!$;
- translation by a complete length-two block multiplies the spline by the Thue–Morse sign of the block;
- the piecewise polynomial pieces glue with the required derivative relations.

The block-translation theorem illustrates the finite-sum strategy. Split a long prefix into complete blocks of size 2^{p+1} plus a remainder. Complete blocks vanish because a degree-p polynomial is below the block's cancellation order. On the remainder, binary concatenation factors the sign, and a change of variables identifies the original spline.

14.7 Probability and spline convergence meet

The import of `ProbabilityRepresentation.lean` gives a canonical limiting CDF/density target. The discrete Toeplitz theorem then combines inner spline convergence with the q-binomial outer row. The result is not simply a pointwise formula pulled from symbolic computation: every floor convention, row normalization, total-variation estimate, and complex translation is certified independently.

15

Legendre coefficients, uniform series, and least squares

15.1 A generic orthogonal-polynomial foundation

The Legendre branch begins below the Fabius-specific layer. `LegendrePolynomial.lean` develops the polynomial identities needed by the project: parity, differential equation, endpoint behavior, derivative bounds, and integral formulas. `LegendreSeriesConvergence.lean` proves a generic smooth-function convergence theorem rather than assuming a prepackaged Hilbert basis result with mismatched normalizations.

The orthogonality proof uses the Sturm–Liouville equation

$$\left((1-x^2)P_n'(x)\right)' + n(n+1)P_n(x) = 0.$$

Multiply the equations for P_m and P_n crosswise, subtract, and integrate. Boundary terms vanish because $1-x^2=0$ at ± 1. If $m \neq n$, the remaining scalar factor is nonzero, forcing

$$\int_{-1}^{1} P_m(x)P_n(x)\,dx = 0.$$

The diagonal integral is proved with the project's normalization, giving the usual coefficient factor $(2n+1)/2$.

15.2 Uniform convergence for smooth functions

For a sufficiently smooth f, repeated integration by parts against the Sturm–Liouville operator yields decay of Legendre coefficients. The code also proves $|P_n(x)| \le 1$ on $[-1,1]$. Absolute summability of coefficients then implies uniform convergence by the Weierstrass M-test.

The generic theorem is stronger than pointwise convergence: it provides a `HasSum` statement in the uniform norm and corresponding pointwise and `tsum` corollaries. This strength pays off in the Fabius specialization, where exact coefficient formulas can be substituted without redoing convergence.

15.3 Fabius coefficients through exact moments

`FabiusLegendreCoefficients.lean` defines the coefficients of up. Since upis even and P_n has parity $(-1)^n$, every odd coefficient is zero. For P_{2n}, expand the polynomial into even monomials:

$$P_{2n}(x) = \sum_{j=0}^{n} p_{n,j} x^{2j}.$$

Then

$$\int_{-1}^{1} \mathrm{up}_F(x) P_{2n}(x)\,dx = \sum_{j=0}^{n} p_{n,j} c_j,$$

and `AnalyticMoments.lean` replaces each integral moment by the exact rational `moment j`. A further rewrite expresses c_j through inverse-dyadic Fabius values when the paper-facing formula prefers function values to moment symbols.

The result is an exact finite coefficient formula. No numerical quadrature or orthogonality approximation appears anywhere in the derivation.

15.4 Even-only and full series

`FabiusLegendreSeries.lean` first applies the generic smooth convergence theorem to `rvachevUp`. It then proves the odd summands vanish and rewrites the full series as an even-index series. The exported API includes:

- absolute summability of the coefficient-weighted Legendre terms;
- pointwise `HasSum` and `tsum` identities;
- uniform convergence on $[-1, 1]$;
- exact coefficient substitutions in terms of moments or dyadic values;
- a translated series for the bounded Fabius function.

`FabiusTranslatedLegendreSeries.lean` transports the expansion under the affine relation between F on $[0, 1]$ and upon $[-1, 1]$. The change of variables is simple on paper but explicit in Lean: domains, Jacobian factors, and the chosen branch of `rvachevUp` all have to line up.

15.5 Least squares as an orthogonal projection theorem

`FabiusLegendreLeastSquares.lean` develops a generic finite projection polynomial

$$\Pi_N f = \sum_{n=0}^{N} a_n P_n.$$

The residual is orthogonal to every polynomial of degree at most N. For any competitor q in that degree class,

$$\|f - q\|_2^2 = \|f - \Pi_N f\|_2^2 + \|q - \Pi_N f\|_2^2.$$

The Pythagorean identity gives minimality and uniqueness without invoking abstract Hilbert-space projection machinery whose polynomial subspace setup would be heavier than the concrete finite basis.

For up, the even partial sum through P_{2N} is already orthogonal to every odd polynomial. Consequently it minimizes error not only among polynomials of degree $2N$, but among all polynomials of degree at most $2N+1$. This "one extra degree for free" is a precise exploitation of symmetry.

Proof idea

The specialized least-squares theorem is assembled from two independent orthogonalities: Legendre orthogonality handles the even basis directions, and function parity kills every odd direction. The stronger degree bound is not a new approximation estimate; it is a

subspace decomposition fact.

15.6 A clean example of generic/specific layering

The Legendre branch is a model for extending the project:

1. prove the reusable spectral theorem with no Fabius imports;
2. specialize only the hypotheses—smoothness, support, parity;
3. compute coefficients through an existing exact bridge;
4. add a thin paper-facing or application-facing facade.

This keeps a future orthogonal-polynomial application from inheriting Fabius specifics, while letting the Fabius result benefit from exact arithmetic that a generic theorem could not know about.

16

The asymptotic tower begins with an exact product

16.1 Why the asymptotic proof does not start with an asymptotic formula

The small-x behavior of the Fabius function is controlled through a negative Laplace product. The formalization's first move is exact. If G denotes the relevant half-moment generating function, iteration of its dyadic functional equation gives, for $s > 0$,

$$G(-s) = \prod_{j \geq 1} \frac{1 - e^{-s/2^j}}{s/2^j}.$$

`NegativeLaplace.lean` constructs the logarithm of this product directly as an absolutely convergent real series. No Mellin inversion, contour movement, or saddle approximation is needed at this stage.

The individual logarithmic factor is

```
noncomputable def negativeLaplaceKernel (x : ℝ) : ℝ :=
  Real.log ((1 - Real.exp (-x)) / x)
```

and the nth dyadic term is the kernel at $s/2^{n+1}$. The elementary bounds

$$-x \leq \log \frac{1 - e^{-x}}{x} \leq 0, \qquad \left| \log \frac{1 - e^{-x}}{x} \right| \leq x$$

produce a geometric majorant and absolute summability.

16.2 The exact dilation equation

Shifting the dyadic index proves

$$L(2s) = K(s) + L(s),$$

where L is `negativeLaplaceLog` and K is `negativeLaplaceKernel`. This one-step relation is the source of the quadratic logarithmic main term and the period-one fluctuation on the $\log_2 s$ scale.

Instead of solving the difference equation only asymptotically, the file adds a forward logarithmic tail

$$T(s) = \sum_{n \geq 0} \log(1 - e^{-s2^n}).$$

Its shift equation cancels the non-polynomial part of $K(s)$. The remaining combination of $L(s)$, $T(s)$, and an explicit quadratic in $\log s$ is exactly invariant under $s \mapsto 2s$. Therefore it is a one-periodic function of $t = \log_2 s$.

The result has the exact form

$$L(s) = -\frac{(\log s)^2}{2\log 2} + \frac{\log s}{2} + R(\log_2 s) + E(s),$$

where $R(t+1) = R(t)$ and the forward-tail error E is positive and satisfies

$$|E(s)| \le \frac{e^{-s}}{(1-e^{-s})^2} \le 4e^{-s} \qquad (s \ge \log 2).$$

The second bound is intentionally simple; the first is the exact quantitative input.

Proof idea

Dyadic periodicity is not an error term. It is the homogeneous solution of an exact dilation equation. Once the forward tail removes the exponentially small defect, the residual is literally periodic. Any later asymptotic formula that discards it must therefore justify why its amplitude vanishes; here it does not.

16.3 From the product logarithm back to the transform

The logarithmic series is exponentiated only after positivity of every factor and convergence have been established. This avoids branch ambiguity in complex logarithms at the base layer. Later complex vertical-line modules build a compatible analytic logarithm near the saddle, but the real exact identity remains the normalization reference.

`LaplaceTransform.lean`, `LaplaceMoments.lean`, and `FabiusComplexMGF.lean` connect the product to the actual transform of the Fabius/Rvachev objects. The separation is useful:

- `NegativeLaplace.lean` can prove exact product algebra with elementary real analysis;
- the transform modules carry integrability, complexification, and differentiation under the integral;
- the saddle modules import a ready-made exact exponent rather than reconstructing the product.

16.4 The asymptotic dependency tower

16.5 Why there are many small asymptotic modules

A saddle proof contains qualitatively different tasks: exact identities, complex differentiability, local Taylor bounds, Gaussian moment algebra, minor-arc decay, phase selection, and asymptotic transfer between variables. Putting all of them in one file would make imports cyclic and theorem statements unreadable. The module granularity records the proof obligations:

`NegativeLaplaceVertical*.lean`
control the complex logarithm and its ordinary or logarithmic jets on vertical lines.

`GaussianPolynomial*.lean`
evaluate or bound Gaussian integrals with polynomial corrections.

`FabiusSaddleReference*.lean`
: define the normalized reference weight and control its tails.

`FabiusSaddleCentral*.lean`
: compare the true central integrand with its finite expansion.

`FabiusSaddleTail*.lean`
: prove the complement is negligible, first quantitatively and then to every order.

`SaddleExpansion*.lean`
: provide generic algebra for finite asymptotic series, logarithms, and flat additions.

The final modules can then state a sharp theorem in a few lines because every analytic estimate has a named interface.

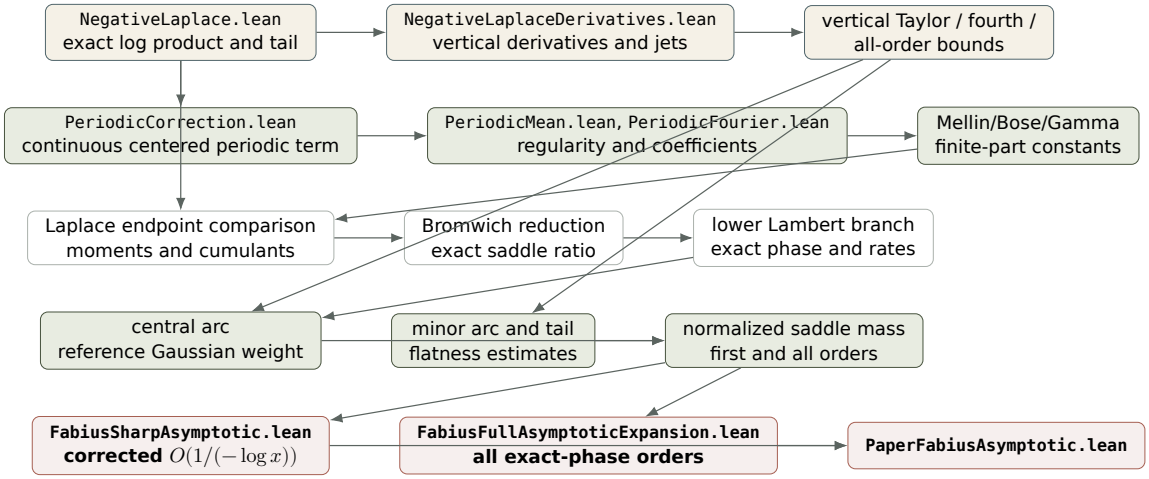

Figure 16.1: The corrected asymptotic tower. Several boxes stand for families of small modules; the direction is exact algebra → regularization → quantitative saddle estimates → source-facing theorems.

17

The periodic correction and its Mellin analysis

17.1 Centering the exact periodic term

The raw periodic correction from `NegativeLaplace.lean` is initially a value defined from convergent series at positive arguments. `PeriodicCorrection.lean` proves local uniform convergence on compact positive intervals using explicit Weierstrass majorants. Continuity in s transfers to continuity in $t = \log_2 s$, and the exact dilation law gives period one.

The module defines a centered function `negativeLaplacePsi` by subtracting the period mean:

$$\int_0^1 \Psi(t)\,dt = 0, \qquad \Psi(t+1) = \Psi(t).$$

Centering is not just cosmetic. It makes the constant part of the asymptotic main term canonical and places all oscillation in a zero-mean function whose Fourier coefficients are unambiguous.

17.2 Mean, smoothness, and Fourier coefficients

The next modules divide regularity questions by strength.

`PeriodicMean.lean`
: computes or identifies the mean and proves the integral manipulations needed to center the correction.

`PeriodicRegularity.lean`
: differentiates the periodized series with uniform bounds on compact sets.

`PeriodicSmooth.lean`
: packages C^∞ regularity.

`PeriodicFourier.lean`
: proves Fourier coefficient formulas, convergence of the Fourier series at the needed strength, and nonconstancy consequences.

The separation matters because the sharp $O(1/t)$ theorem only needs bounded regularity at certain stages, while the obstruction theorem needs a certified nonzero oscillatory component.

17.3 Mellin regularization

The Fourier coefficients of a log-dyadic periodization naturally involve a Mellin transform. The zero frequency is singular and must be treated as a finite part. The files `MellinBose.lean`,

`BoseFinitePartIntegral.lean`, and `MellinFinitePart.lean` formalize this without hiding the pole cancellation.

Near $s = 0$, the product $\Gamma(s)\zeta(1 + s)$ has a double pole. The code subtracts the principal part and defines a regular product whose limit is the constant `gammaZetaConstant`. It proves both a complex punctured limit and the real one-sided limit used in the final main term. Auxiliary modules `GammaSecondOrder.lean` and `StieltjesConstant.lean` provide the exact second-order expansions and the chosen first Stieltjes constant convention.

Lean engineering

The constant is carried as a named exact expression rather than a decimal. This prevents sign-convention mistakes for the first Stieltjes constant and allows the compact Lambert formula to be algebraically expanded into the literal Euler–Stieltjes form only in the dedicated Wikipedia-expansion module.

17.4 The periodic term is genuinely nonconstant

The Fourier analysis proves that at least one nonzero Fourier coefficient of `negativeLaplacePsi` is nonzero. Therefore Ψ cannot be absorbed into the constant. Since its phase tends to infinity while advancing through all residue classes modulo one, it contributes bounded but persistent oscillation to $\log F(x)$.

This is the decisive fact behind the correction. A remainder bounded by $C/(-\log x)$ tends to zero. A nonconstant periodic function sampled at the exact phase does not. Therefore an elementary main term omitting Ψ cannot have the claimed inverse-logarithmic error.

17.5 Endpoint Laplace comparison

The transform asymptotic must eventually become a statement about the function near its endpoint. `EndpointLaplaceComparison.lean` relates the Laplace integral to a local endpoint mass and controls the difference. Positivity, monotonicity, and compact support allow upper and lower bounds that isolate the same saddle scale. `LaplaceMomentBounds.lean` and `LaplaceCumulantAsymptotics.lean` supply derivative information and convexity for the logarithmic transform.

This comparison is an important conceptual bridge: the product describes a transform globally, while the theorem concerns $F(x)$ locally as $x \to 0^+$. The formalization does not silently identify the two; it proves an exact or quantified transfer theorem and names the residual ratio.

18

The lower Lambert phase and the corrected sharp asymptotic

18.1 The exact phase equation

The saddle coordinate is not the naive dyadic logarithm $t = -\log_2 x$. It is the lower-Lambert solution λ of

$$\lambda 2^{-\lambda} = x.$$

Equivalently,

$$\lambda - \log_2 \lambda = t.$$

The file `LowerLambertW.lean` constructs the lower real branch by inverting $u \mapsto u \log u$ on the appropriate interval, proves the equation and branch inequalities, and packages the paper's solution `paperLambertN`.

For the small-argument variable,

$$\lambda(x) = -\frac{W_{-1}(-x \log 2)}{\log 2}.$$

The declaration is `fabiusLambertPhase`. On the dyadic logarithmic scale, `dyadicLambertPhase t` is the same quantity at $x = 2^{-t}$. `FabiusLambertPhase.lean` proves

$$\lambda(t) = t + \log_2 t + o(1)$$

and refines this to the quantitative remainders needed by the literal $O(1/t)$ expansion.

Proof idea

Keeping λ exact makes the saddle stationary by definition. Replacing it too early by $t + \log_2 t$ shifts the phase of the periodic correction. Even a small absolute phase error can create an error larger than $1/t$ after a nonconstant periodic function is composed with it.

18.2 The compact corrected main term

The cleanest source-facing expression is defined in `FabiusWikipediaMain.lean`. Put

$$W = W_{-1}(-x \log 2), \qquad \lambda = -W/\log 2.$$

Then

$$M_{\mathrm{corr}}(x) = -\frac{7\log 2}{12} - \frac{1}{2}\log(\pi x) + \frac{\gamma_* - W - \frac{1}{2}W^2}{\log 2} + \Psi(\lambda),$$

where γ_* is `gammaZetaConstant` and Ψ denotes `negativeLaplacePsi`. In Lean the displayed expression is `fabiusCorrectedWikipediaMain`. The file proves it is exactly equal to the internal saddle main `fabiusSharpLambertMain` whenever the lower Lambert branch lies in its natural domain.

The equality uses only the exact saddle equation. Taking logarithms of $\lambda 2^{-\lambda} = x$ gives

$$\log x = \log\lambda - (\log 2)\lambda,$$

which converts the transform-side exponent to the compact W expression.

18.3 The literal elementary expansion

`FabiusWikipediaExpansion.lean` expands the compact main into powers of $\log x$ and $\log(-\log x)$. It defines `fabiusWikipediaElementaryMain`, the nonperiodic expression appearing in the source discussion, including its Euler–Stieltjes constant and the visible $1/\log x$ correction. The corrected literal main is

$$M_{\mathrm{explicit}}(x) = M_{\mathrm{elementary}}(x) + \Psi(\lambda(x)).$$

On $x = 2^{-t}$, its nonperiodic component is organized as

$$-\frac{\log 2}{2}t^2 - t\log t + \left(1 + \frac{\log 2}{2}\right)t - \frac{(\log t)^2}{2\log 2} + C - \frac{(\log t)^2}{2(\log 2)^2 t},$$

plus an explicit source-conversion term of order $1/t$. A fourth phase term from `FabiusLambertHigherExpansion.lean` is needed: stopping one refinement earlier would leave $O((\log t)/t)$, not $O(1/t)$.

18.4 Central arc, minor arc, and normalized mass

The saddle proof is split into a central region and its complement.

Reference weight.
`FabiusSaddleReferenceWeight.lean` defines the Gaussian-normalized measure after centering at the exact saddle. Its total mass and polynomial moments are controlled by the `GaussianPolynomial*.lean` files.

Central comparison.
`FabiusSaddleCentral.lean` and `FabiusSaddleCentralLambert.lean` Taylor-expand the vertical logarithm, exponentiate the controlled remainder, and compare the true central integrand with the reference Gaussian weight.

Minor arc and tail.

`NegativeLaplaceMinorArc.lean`, `FabiusLambertMinorArc.lean`, `FabiusSaddleTail.lean`, and reference-tail modules show that away from the saddle the real part drops uniformly. The tail is small at the quantitative order needed for the sharp theorem.

Exact reduction.

`FabiusSharpExactReduction.lean` expresses $\log F(x) - M_{\mathrm{corr}}(x)$ as the logarithm of a normalized saddle ratio plus the exponentially small forward-tail error.

The headline analytic estimate is that the normalized saddle kernel mass is $1 + O(1/\lambda)$. Taking a logarithm preserves the order because the ratio is eventually positive and tends to one.

18.5 The corrected theorem and the obstruction theorem

`FabiusSharpAsymptotic.lean` exposes three complementary statements. For any `F` satisfying `IsFabius`,

$$\log F(x) - M_{\mathrm{explicit}}(x) = O\left(\frac{1}{-\log x}\right) \qquad (x \to 0^+).$$

This is `log_fabius_sub_explicitCorrectedWikipediaMain_isBigO`. The compact Lambert form has an analogous theorem.

The file also proves the negative statement

$$\log F(x) - M_{\mathrm{elementary}}(x) \neq O\left(\frac{1}{-\log x}\right),$$

through `log_fabius_sub_WikipediaElementaryMain_not_isBigO`. The proof uses nonconstancy of Ψ and sequences of exact phases approaching two points with different periodic values. Thus the missing term is not optional at the claimed precision.

Finally, exponentiating a logarithmic error tending to zero gives

$$F(x) \sim \exp(M_{\mathrm{explicit}}(x)).$$

Lean proves this by rewriting the quotient as the exponential of the log difference, using strict positivity of $F(x)$ for $x > 0$, and composing the error limit with continuity of `Real.exp`.

Formalization-driven correction

The formal result is stronger than replacing a wrong constant. It identifies a missing *nonconstant periodic function*, fixes the phase at the exact lower-Lambert coordinate, proves the repaired inverse-logarithmic error, and proves that the uncorrected claim fails at that same rate.

18.6 Why a quotient-of-exponentials fit is not an asymptotic equivalent

`FabiusQuotientExponentialMismatch.lean` analyzes a visually effective compact-interval fit built from a quotient of exponentials. Its endpoint decay is of $e^{-c/x}$ type. The Fabius function decays faster than every power but slower than $e^{-c/x}$ for every fixed $c > 0$. Therefore the

quotient fit tends to zero too rapidly and cannot be an asymptotic equivalent, even if it is numerically attractive away from the endpoint.

The qualitative comparison is formalized separately in `FabiusFlatness.lean` and `FabiusDecayComparison.lean`; the mismatch module then turns those rates into a quotient or non-equivalence theorem.

19

The full exact-phase asymptotic expansion

19.1 A generic notion of expansion

The all-orders branch does not encode each truncation as an unrelated theorem. The `SaddleExpansion*.lean` files define a reusable `HasAsymptoticExpansion` interface. Given a filter $\mathcal{F}$, a small scale $\varepsilon(x)$, a target $f(x)$, and coefficient functions $a_j(x)$, the structure records:

- every coefficient term has the expected Big-O size;
- for every N, the difference between f and $\sum_{j<N} \varepsilon^j a_j$ is $O(\varepsilon^N)$.

The coefficients are permitted to depend on x. This is essential here: they are periodic functions evaluated at the exact Lambert phase, not fixed constants.

The generic library proves closure under finite sums, products, scalar operations, addition of a flat function, and composition with a logarithm near one. Separate algebra modules handle coefficient convolution and the formal logarithm recurrence.

19.2 Vertical jets and finite exponential algebra

The complex saddle exponent is Taylor-expanded in the vertical variable. The modules

- `NegativeLaplaceVerticalOrdinaryJets.lean`,
- `NegativeLaplaceAllOrderJets.lean`,
- `NegativeLaplaceVerticalAllOrderBound.lean`, and
- `FabiusSaddleExponentAllOrders.lean`

provide an arbitrary finite jet together with a uniform remainder on the central arc.

Exponentiating a truncated polynomial produces coefficients indexed by partitions of the total order. Rather than asking ring normalization to rediscover this combinatorics in every theorem, `SaddleExpansionAlgebra.lean`, `SaddleExpansionFiniteRemainder.lean`, and `FabiusSaddlePolynomialCoefficients.lean` define the coefficient recursion and prove the finite remainder estimate abstractly.

19.3 Gaussian contraction of polynomial corrections

After scaling, odd Gaussian moments vanish and even moments are explicit. The files `GaussianPolynomialContraction.lean`, `GaussianPolynomialWholeIntegral.lean`, `GaussianPolynomialTail.lean`, and `GaussianPolynomialTailAllOrders.lean` convert the exponent coefficients into mass coefficients. They also prove that replacing a finite central interval

by the whole Gaussian line produces an error flat to every requested order once the central radius grows appropriately.

The output is `fabiusSaddleMassCoefficient j lam`, a real coefficient function of the exact phase λ. Its dependence on λ contains the periodic derivative data inherited from the negative Laplace correction. The zeroth coefficient is one.

19.4 Central and tail estimates to every order

`FabiusSaddleCentralAllOrders.lean` proves that the central kernel mass has the finite expansion with a remainder of order λ^{-N}. `FabiusSaddleTailAllOrders.lean` proves that the omitted saddle contour is flat compared with every such power. `FabiusSaddleMassAllOrders.lean` adds the pieces and packages a single expansion theorem.

The distinction between "central remainder" and "tail flatness" is important. The central error is produced by truncating an analytic Taylor series and must be tracked at exactly order N. The tail is exponentially or superpolynomially small and can be added through a generic `add_flat` theorem without changing any coefficient.

19.5 From complex kernel mass to a real saddle ratio

The kernel analysis is naturally complex, but the Fabius saddle ratio is real. `FabiusFullAsymptoticExpansion.lean` first proves that complexification of the real ratio equals the kernel mass. Norm compatibility then transfers both coefficient and remainder estimates:

```
theorem fabiusSaddleRatio_dyadicLambert_hasAsymptoticExpansion
    (F : BoundedFabius) (hF : IsFabius F) :
    HasAsymptoticExpansion atTop
      (fun t : ℝ => (dyadicLambertPhase t)⁻¹)
      (fun t : ℝ => fabiusSaddleRatio F ((2 : ℝ) ^ (-t))
        (fabiusLambertRadius ((2 : ℝ) ^ (-t)))
        (dyadicLambertPhase t))
      (fun j t =>
        fabiusSaddleMassCoefficient j (dyadicLambertPhase t)) := by
  ...
```

This theorem is the all-orders analogue of the first-order statement that the normalized mass equals $1 + O(1/\lambda)$.

19.6 Taking the logarithm

The desired expansion is for $\log F$, so the ratio expansion must pass through `Real.log`. The generic method `HasAsymptoticExpansion.real_log` requires:

- the small scale tends to zero;
- the zeroth mass coefficient is identically one;
- the target ratio is eventually positive.

All three are proved explicitly. The logarithmic coefficient recurrence is packaged as `fabiusSaddleLogCoefficient`. The zeroth log coefficient vanishes, as it should for the logarithm of a series beginning with one.

The exponentially flat forward-tail error from `NegativeLaplace.lean` is then added with the generic flat-addition theorem. This is a particularly clean payoff of the exact decomposition: the forward tail never enters the coefficient recursion.

19.7 Finite truncations and the first explicit correction

Define

$$S_N(\lambda) = \sum_{j=0}^{N-1} \lambda^{-j} c_j(\lambda)$$

after logarithmic coefficient conversion, and

$$M_N(x) = M_{\mathrm{sharp}}(x) + S_N(\lambda(x)).$$

These are `fabiusSaddleLogPartialSum` and `fabiusSharpLambertExpansion`. For every N,

$$\log F(x) - M_N(x) = O(\lambda(x)^{-N}).$$

Since $\lambda(x)$ is comparable to $-\log x$, the file also exports the weaker but more elementary rate

$$O((-\log x)^{-N}).$$

At $N = 2$, the zeroth term vanishes and the first correction is explicit:

$$S_2(\lambda) = \frac{\mathrm{fabiusFirstSaddleCorrection}(\lambda)}{\lambda}.$$

The coefficient is periodic/phase-dependent; the theorem identifies it but does not assert it is nowhere zero.

19.8 Transport from the dyadic logarithmic scale to $x \to 0^+$

Most quantitative work is done with $x = 2^{-t}$ and $t \to +\infty$, where dyadic periodicity and the Lambert phase are natural. The generic module `SaddleExpansionSmallArgument.lean` proves a filter-transfer theorem between these coordinates. `SaddleLogAsymptoticTransfer.lean` handles the logarithmic form and scale comparison. The final theorem is:

```
theorem log_fabius_sub_sharpLambertMain_hasAsymptoticExpansion
    (F : BoundedFabius) (hF : IsFabius F) :
    HasAsymptoticExpansion (nhdsWithin 0 (Set.Ioi 0))
      (fun x : ℝ => (fabiusLambertPhase x)⁻¹)
      (fun x : ℝ => Real.log (fabiusReal F x) -
        fabiusSharpLambertMain x)
      (fun j x => fabiusSaddleLogCoefficient j
        (fabiusLambertPhase x)) := by
  ...
```

The finite-sum corollary `log_fabius_sub_sharpLambertExpansion_isBigO` is usually the most convenient consumer API.

19.9 What is and is not expanded

The phase itself is developed to arbitrary order in `FabiusLambertAllOrderAlgebra.lean`, `FabiusLambertAllOrderRemainder.lean`, `FabiusLambertAllOrderSmallArgument.lean`, and related higher-expansion modules. Nevertheless the final saddle coefficients remain evaluated at the *exact* phase $\lambda(x)$.

Substituting an asymptotic expansion for λ inside every oscillatory coefficient $c_j(\lambda)$ is a separate composition problem. One would need all-order Taylor control of periodic coefficient functions and careful bookkeeping of phase perturbations. The source comments explicitly decline to claim that additional theorem. This is mathematical precision, not an unfinished proof: the exact-phase expansion is already a full expansion at all orders in $1/\lambda$.

Lean engineering

The all-orders theorem is parameterized by coefficient functions rather than a single formal series with constant coefficients. That design exactly matches log-periodic asymptotics. Forcing constant coefficients would either lose the oscillation or bury it in a nonstandard generalized coefficient ring.

19.10 Dyadic and q-binomial sharp transfers

A parallel family—`DyadicSharpDecomposition.lean`, `DyadicSharpConditional.lean`, `FabiusDyadicSharpCumulant.lean`, `FabiusDyadicSharpAsymptotic.lean`, and the q-binomial sharp-transfer modules—specializes the small-argument theory to exact dyadic sequences or finite formulas. The strategy is to insert an exact dyadic representation before taking the asymptotic limit. This permits statements about exact rational sequences whose logarithms inherit the same Lambert/periodic behavior.

The conditional files separate a generic asymptotic implication from the specific analytic estimates needed to discharge it. That structure makes it possible to experiment with alternate discrete decompositions without rebuilding the saddle proof.

20

Recurring Lean proof-engineering patterns

20.1 Carry invariants in types when they are truly structural

`BoundedFabius` stores the range $[0, 1]$ in the codomain. The fixed-point space stores symmetry as a subtype condition. Probability coordinates use `Set.Icc 0 1`. These choices move ubiquitous obligations from theorem hypotheses into projections from the type.

The project does not apply this maxim indiscriminately. Compact support, for example, remains a proposition rather than a bespoke function type because it is proved for several derived functions and consumed by standard mathlib APIs. The practical rule is: put an invariant in the type when almost every operation must preserve it and there is a natural complete/algebraic structure on the subtype.

20.2 Prove exact algebra before casting

Moment recurrences, Thue–Morse sums, q-binomial identities, and dyadic values are proved over $\mathbb{N}$, $\mathbb{Z}$, or $\mathbb{Q}$. Cast the final theorem to $\mathbb{R}$ or $\mathbb{C}$. This preserves decidable equality, normalization, parity, divisibility, and kernel computation. The bridge lemmas are then explicit and searchable, usually with names ending in `_cast`.

20.3 Separate implementation invariants from mathematical specifications

The dyadic evaluator is the clearest example:

- table length, prefix stability, and recursion decrease are program properties;
- block Taylor and bit recurrence are mathematical properties;
- a generic strong-induction theorem connects them.

This makes refactoring possible. A vector-backed or memoized table could reuse the mathematical recurrence after reproving only the implementation interface.

20.4 Use local finiteness instead of unnecessary convergence theory

The signed global extension is written as a `tsum`, but every local proof reduces it to a finite sum by support. The same principle appears in partitions of unity and translated bump sums. When a series is locally finite, proving absolute convergence of every derivative is both harder and less informative than proving a local finite support set.

20.5 Package smooth compact support as Schwartz only at transform boundaries

The code does not make upa Schwartz map at its definition site. It first proves smoothness and support in ordinary function language. Modules that need Fourier injectivity or Poisson summation then package those facts into a Schwartz object. This avoids forcing all elementary lemmas through coercions from a heavyweight bundled type.

20.6 Strong induction follows binary control flow

Binary algorithms often recurse neither on the scale alone nor on the immediate predecessor. Clearing the highest set bit produces an arbitrary smaller natural. `Nat.strong_induction_on` matches the program and avoids inventing an artificial measure. The same style appears in coefficient uniqueness arguments where the nth coefficient depends on all earlier ones.

20.7 Use formal power series for coefficient identities

Analytic power-series proofs carry convergence radii and topology even when the claim is purely finite coefficient algebra. The moment and q-binomial branches use formal series until a theorem genuinely needs evaluation at a real or complex argument. Coefficient extraction then turns multiplication into a finite convolution with no summability side conditions.

20.8 Normalize before proving parity or divisibility

A rational recurrence obscures parity. Clearing a carefully chosen common denominator creates a natural recurrence where “all terms but one are even” is visible to `omega`, `norm_num`, and divisibility lemmas. The normalized sequence is not auxiliary clutter; it is the correct domain for the theorem.

20.9 Factor generic convergence arguments away from special coefficients

The interval-average squeezing theorem, finite-row Toeplitz theorem, generic Legendre convergence theorem, generic least-squares projection theorem, and generic saddle-expansion algebra all follow this pattern. The special branch proves a small interface—monotonicity, bounded variation, smoothness, or a finite jet—and the generic theorem handles the limiting logic.

20.10 Prefer exact cancellation to estimation when symmetry allows it

Taylor reduction cancels two remainders rather than bounding them. Complete Thue–Morse blocks annihilate low-degree polynomials exactly. Odd Legendre coefficients vanish exactly. The negative-Laplace dilation equation is solved with an exact periodic correction before the remaining tail is estimated. Exact symmetry usually yields stronger theorems and simpler downstream error bookkeeping.

20.11 Make false source claims first-class

The repository uses counterexample and obstruction modules instead of merely editing comments. A negative theorem such as `log_fabius_sub_WikipediaElementaryMain_not_isBigO` is a durable API: later work cannot accidentally assume the discarded statement, and a reader can see precisely which quantifier or rate fails.

20.12 Control coercions at named boundaries

Common coercion seams include:

- subtype value $\to \mathbb{R}$ through `fabiusReal`;
- $\mathbb{Q} \to \mathbb{R}$ through exact `_cast` theorems;
- real functions $\to$ complex functions for Fourier/Laplace analysis;
- real ratios $\to \mathbb{C}$ kernel masses in the saddle proof;
- natural floor lengths $\leftrightarrow$ integer floor endpoints in discrete sums.

Each seam receives a lemma. This is preferable to repeated local `norm_cast` improvisation, which tends to make proofs brittle under normal-form changes.

20.13 Keep endpoint facts explicit

Many informal proofs say "the boundary term vanishes." Here the reason may be constant tails, compact support, flatness at zero, flatness at a power of two, or the factor $1 - x^2$ in the Legendre operator. The code records each as a separate theorem and invokes exactly the right one in integration by parts or gluing. This makes corrections to endpoint conventions local.

21 Task-oriented reading paths and extension points

21.1 To understand the canonical construction

Suggested reading path

Read `Basic.lean`, the first half of `Differential.lean`, and then `Existence.lean`. Keep mathlib documentation for complete metric spaces, continuous maps on compact sets, Bochner/interval integrals, and contraction maps nearby. After the fixed point is built, return to the smoothness bootstrap in `Differential.lean` and finally read the uniqueness tail of `Existence.lean`.

The main extension point is an alternative construction: prove the resulting object satisfies `IsFabius`; the uniqueness API supplies equality with the canonical function.

21.2 To modify or optimize the dyadic evaluator

Suggested reading path

Read `Arithmetic.lean`'s evaluator definitions, then all of `DyadicCorrectness.lean`. Treat `FabiusDyadicHasBitRecurrence` as the semantic interface. Read `DyadicClosedForm.lean` only after understanding which recurrence the implementation needs, and `DyadicAnalytic.lean` last.

A performance refactor should preserve the public exact functions or provide conversion theorems. Likely reusable pieces are table-prefix stability, representation refinement, and the strong-induction correctness theorem.

21.3 To add a new exact arithmetic theorem

Start from `MomentPowerSeries.lean`, `NormalizedMoments.lean`, and `Arithmetic.lean`. Decide first whether the claim belongs to $\mathbb{Q}$, normalizing naturals, or a real cast. Prove the strongest exact statement in the smallest type, then add a cast theorem near the existing analytic bridge. For parity or valuation, introduce a division-free normalization before attempting induction.

21.4 To follow the first paper

Read `Paper05442.lean` with the paper open, jumping to:

- `OriginalCharacterization.lean` and `OriginalUniqueness.lean` for Theorem 1;

- `EarlyApproximants.lean`, `StepApproximationLimit.lean`, and `WeakConvergence.lean` for the finite approximants;
- `ProbabilityRepresentation.lean` for the random-series model;
- `AnalyticMoments.lean` and `FourierProduct.lean` for moments and transform;
- `PoissonSummation.lean` for lattice identities.

Read `OriginalPaperSupplement.lean` whenever a displayed equation is not represented by a numbered theorem in the facade.

21.5 To follow the arithmetic paper

Read `Paper06487.lean` for the numbered map, then `Paper06487Supplement.lean`. The mechanism-oriented path is

`GlobalExtension.lean` ⟶ `ScaleTranslation.lean` ⟶ `TaylorReduction.lean` ⟶ `GlobalDyadic.lean` ⟶ `DenominatorBound.lean` ⟶ `TwoAdic.lean`.

For exact inverse powers, detour through `MomentPowerSeries.lean` and `AnalyticMoments.lean`.

21.6 To work on q-binomial or discrete formulas

Begin with `HalfQBinomial.lean` and the `ThueMorse*.lean` modules. Then read the raw q-binomial formula before the simplified facade. For limits, read `FabiusDiscreteLimitToeplitz.lean`; its generic theorem tells you the three properties a new outer row must satisfy. The spline module is the best worked example of binary block cancellation with a floor-defined prefix.

21.7 To work on spectral approximation

Read `LegendrePolynomial.lean` and `LegendreSeriesConvergence.lean` without Fabius context first. Then `FabiusLegendreCoefficients.lean` shows how exact moments enter. Finally read `FabiusLegendreSeries.lean` and `FabiusLegendreLeastSquares.lean`. A new orthogonal basis should follow the same pattern: generic convergence/orthogonality below, exact Fabius coefficient bridge above.

21.8 To work on the asymptotic theory

Suggested reading path

Do not start with `FabiusSharpAsymptotic.lean`. Read `NegativeLaplace.lean`, `PeriodicCorrection.lean`, and `FabiusWikipediaMain.lean` first. Then study `FabiusSharpExactReduction.lean` to see the exact identity that the saddle estimate must close. For first order, follow the central/tail Lambert modules. For all orders, learn the generic `SaddleExpansion*.lean` API before reading `FabiusSaddleMassAllOrders.lean` and `FabiusFullAsymptoticExpansion.lean`.

A new refinement should preserve the exact phase until the final transfer. If substituting an elementary phase expansion inside periodic coefficients, state and prove a composition theorem rather than relying on informal smoothness.

21.9 Where to place a new theorem

A practical placement rule is:

Generic mathematical lemma
: goes in the lowest module with no unnecessary Fabius imports.

Fabius bridge between two established models
: gets its own focused module or joins the existing bridge module.

Exact executable or rational identity
: belongs below analytic casts.

Paper-numbered restatement
: belongs in the relevant facade after the mechanism theorem exists.

Correction or refutation
: should be named explicitly and documented in an audit/supplement module, not hidden in a changed formula.

A

Categorized module catalogue

This appendix lists every module under `Lean/FabiusFunction/` at the pinned snapshot. The descriptions state the module's architectural role, not an exhaustive inventory of declarations. The root umbrella `Lean/FabiusFunction.lean` is listed separately after the table.

Module	Principal role
Core specifications, construction, and global calculus	
`Arithmetic.lean`	Exact natural/rational definitions: moments, Thue–Morse signs, dyadic closed forms, evaluator tables, and denominator data.
`Basic.lean`	Core types and predicates, Rvachev folding, signed extension, sinc, and transform-level vocabulary.
`Differential.lean`	Global derivative refinement for the up function and the generic smoothness bootstrap.
`Existence.lean`	Complete symmetric function space, contraction operator, canonical fixed point, and dyadic-density uniqueness.
`GlobalExtension.lean`	Local finiteness, smoothness, agreement on the unit interval, and the global equation F'(x)=2F(2x).
`GlobalDyadic.lean`	Exact rational evaluation of the signed global extension on arbitrary dyadic blocks.
`Parity.lean`	Reusable even/odd identities for functions, coefficients, finite sums, and dyadic decompositions.
`PaperStatements.lean`	Stable canonical declarations and paper-facing core statements shared by aggregate modules.
Moments, recurrences, generating functions, and transforms	
`AnalyticMoments.lean`	Integral moments of the up function, normalization, integration-by-parts recurrences, and generating identities.
`MomentPowerSeries.lean`	Purely formal power-series recurrences and coefficient uniqueness for moments and half moments.
`NormalizedMoments.lean`	Natural numerator and common-denominator normalization for the half-moment sequence.
`NormalizedEvenMoments.lean`	Exact normalization and integrality properties for the even moment sequence.
`HalfMomentDenominator.lean`	Sharper denominator products and divisibility facts for half moments.

Continued on next page

Module	Principal role (continued)
BernoulliRecurrences.lean	Bernoulli-number forms of the moment and inverse-power recurrences.
FabiusRecurrenceSequence.lean	The normalized inverse-power sequence, positivity, rapid decay, recurrences, and generating series.
ExactInversePower.lean	Exact formulas identifying inverse-power Fabius values with normalized moment data.
FourierAnalytic.lean	Analytic/entire properties of sinc products and transform series.
FourierProduct.lean	Convergence of the infinite sinc product and equality with the Fourier transform of the up function.
LaplaceTransform.lean	Laplace transform definitions and identities for the canonical compactly supported density.
LaplaceMoments.lean	Moment expansions and differentiation formulas for the Laplace transform.
LaplaceMomentBounds.lean	Uniform bounds for Laplace moments and derivatives used in saddle estimates.
FabiusComplexMGF.lean	Complex moment-generating function and analytic continuation interfaces.
Certified dyadic evaluation, denominators, and valuations	
DyadicCorrectness.lean	Program invariants and the abstract bit-recurrence theorem certifying the executable evaluator.
DyadicClosedForm.lean	Thue–Morse block cancellation, polynomial/Hasse-derivative identities, and exact bit recurrence.
DyadicAnalytic.lean	Analytic Taylor-block recurrence and cast equality between exact dyadic values and every Fabius solution.
DenominatorBound.lean	Explicit common denominators, integrality after scaling, and LCM bounds for finite dyadic grids.
TwoAdic.lean	Odd normalized numerators and exact 2-adic valuations of inverse-dyadic values.
FabiusInverseDyadicClosedForm.lean	Consolidated inverse-dyadic closed formulas in final source-facing notation.
FabiusDyadicLogBounds.lean	Quantitative logarithmic bounds for exact dyadic values, used in asymptotic comparison.
Original-paper construction, probability, and approximation	
OriginalCharacterization.lean	The compact-support specification with unknown scale and derivation that the scale equals two.
OriginalUniqueness.lean	Schwartz packaging, Fourier refinement, infinite sinc product, and transform-injectivity uniqueness.

Continued on next page

Module	Principal role (continued)
`OriginalPaperSupplement.lean`	Unnumbered displayed equations, endpoint details, and corrections supporting the first paper.
`EarlyApproximants.lean`	Finite convolution measures, characteristic polynomials, and half-endpoint step approximants.
`EarlyMeasureBridge.lean`	Links between finite algebraic approximants and their probability-measure realizations.
`StepMeasureBridge.lean`	Identifies step functions with densities or interval masses of the finite measures.
`StepApproximationLimit.lean`	Unimodality, interval-average squeezing, and pointwise convergence to the up function.
`WeakConvergence.lean`	Characteristic-function convergence and Levy continuity for the finite probability measures.
`ProbabilityRepresentation.lean`	Infinite product probability space, weighted random series, self-similar law, and CDF identification.
`PoissonSummation.lean`	Schwartz rapid decay of every real-axis Fourier derivative, Poisson summation, partitions of unity, period-two expansions, and source corrections.
Translation, Taylor reduction, and arithmetic-paper support	
`ScaleTranslation.lean`	Power-of-two translations, Thue–Morse sign flips, derivative translation, and remainder cancellation.
`TaylorReduction.lean`	Exact finite Taylor reduction across dyadic annuli, including integer-scale branches.
`Paper06487Supplement.lean`	Unnumbered arithmetic-paper claims, monotonicity, flatness, expanded formulas, and repaired proof factors.
Thue–Morse, q-binomial formulas, and discrete limits	
`HalfQBinomial.lean`	Gaussian binomial coefficients at q=1/2, finite Pochhammer products, positivity, and the finite q-binomial theorem.
`ThueMorsePrefix.lean`	Finite signed prefixes, binary block decompositions, and iterated prefix operations.
`ThueMorseGenerating.lean`	Formal and polynomial generating identities for signed Thue–Morse data.
`ThueMorseExponential.lean`	Finite exponential product factorization and consequences for power-sum cancellation.
`ThueMorseApproximation.lean`	Corrected finite approximation theorem and quantitative error estimates.
`ThueMorseBinomialLog.lean`	Zero–one Thue–Morse sequence, binomial parity, and exact Log2 formulas.
`DraftCounterexamples.lean`	Machine-checked counterexamples to false normalizations and error claims in the local K-fold draft.

Continued on next page

Module	Principal role (continued)
FabiusParityPowerSeries.lean	Parity-filtered formal series used to isolate even/odd Fabius coefficients.
FabiusRawQBinomialFormula.lean	Unrearranged exact finite q-binomial/Thue–Morse formula close to direct expansion.
FabiusRawQBinomialScalar.lean	Scalar-field generalization of the raw finite formula.
FabiusQBinomialFormula.lean	Coefficient-extractor proof and simplified exact formulas for inverse powers and arbitrary dyadic numerators.
FabiusQBinomialScalarFormula.lean	Real/complex scalar versions of the simplified q-binomial identity.
FabiusQBinomialTaylor.lean	Taylor and coefficient-isolation interpretation of the q-binomial weights.
FabiusDyadicQBinomialScalar.lean	Exact dyadic specialization of the scalar q-binomial formula.
FabiusGlobalQBinomialSeries.lean	Global signed-series forms and translation consequences of the q-binomial identity.
FabiusBinaryReductionSeries.lean	Binary-block reduction of global formulas into convergent or finite series.
FabiusDiscreteLimitIntegration.lean	Integral and inner-prefix limits underlying the discrete approximation formula.
FabiusDiscreteLimitComplexShift.lean	Real and complex translation parameters in the inner Thue–Morse polynomial limit.
FabiusDiscreteLimitToeplitz.lean	Finite-row Toeplitz theorem, q-binomial row normalization, and bounded total variation.
FabiusUniformSpline.lean	Half-cell-centered Thue–Morse splines, block translation, gluing, and convergence.
FabiusComplexShiftSpline.lean	Complex-shift extension of the spline identities and finite polynomial cancellation.
Legendre and polynomial approximation	
LegendrePolynomial.lean	Legendre recurrence, parity, Sturm–Liouville equation, orthogonality, bounds, and normalization.
LegendreSeriesConvergence.lean	Generic absolute and uniform convergence of Legendre series for smooth functions.
FabiusLegendreCoefficients.lean	Exact even coefficients from moments and inverse-dyadic values; vanishing of odd coefficients.
FabiusLegendreSeries.lean	Pointwise and uniform Legendre expansions of the up function with exact coefficients.
FabiusTranslatedLegendreSeries.lean	Affine transport of the up-function series to the bounded Fabius function.

Continued on next page

Module	Principal role (continued)
`FabiusLegendreLeastSquares.lean`	Generic projection/Pythagorean theorem and the strengthened even Fabius least-squares result.
Negative-Laplace product and vertical analytic control	
`NegativeLaplace.lean`	Absolutely convergent logarithmic product, exact dyadic dilation law, periodic decomposition, and exponential tail.
`NegativeLaplaceDerivatives.lean`	Derivative formulas for the negative-Laplace logarithm and termwise differentiation.
`NegativeLaplaceDerivativeBounds.lean`	Uniform bounds for higher derivatives on positive and saddle-scale regions.
`NegativeLaplaceVertical.lean`	Complex vertical-line evaluation near the real saddle and branch-compatible logarithms.
`NegativeLaplaceVerticalLog.lean`	Analytic logarithm of the vertical product and exact real-axis normalization.
`NegativeLaplaceVerticalSmooth.lean`	Smooth/holomorphic dependence of the vertical logarithm on the displacement variable.
`NegativeLaplaceVerticalTaylor.lean`	Finite vertical Taylor formula with explicit remainder control.
`NegativeLaplaceVerticalFourthBound.lean`	Fourth-order derivative bound used by the first quantitative saddle theorem.
`NegativeLaplaceVerticalAllOrderBound.lean`	Arbitrary-order vertical derivative bounds for the full expansion.
`NegativeLaplaceVerticalOrdinaryJets.lean`	Ordinary derivative jets and coefficient normalization on the vertical scale.
`NegativeLaplaceAllOrderJets.lean`	All-order logarithmic/exponential jet package for saddle expansion.
`NegativeLaplaceMinorArc.lean`	Decay of the product away from the central vertical neighborhood.
`NegativeLaplaceScaledTailFlat.lean`	Superpolynomial flatness of scaled tails at the exact Lambert saddle.
Periodic correction, Mellin finite parts, and Laplace endpoint data	
`PeriodicCorrection.lean`	Continuity, exact period one, centering, and the zero-mean function negativeLaplacePsi.
`PeriodicMean.lean`	Mean-value calculation and integral identities for the raw periodic correction.
`PeriodicRegularity.lean`	Differentiability and uniform derivative convergence of the periodized series.
`PeriodicSmooth.lean`	C-infinity packaging for the centered periodic correction.
`PeriodicFourier.lean`	Fourier coefficients, Fourier-series representation, and nonconstancy of the periodic term.

Continued on next page

Module	Principal role (continued)
MellinBose.lean	Mellin transform of the Bose/logarithmic kernel and nonzero Fourier modes.
MellinFinitePart.lean	Regularization of Gamma(s) zeta(1+s), double-pole subtraction, and the finite-part constant.
BoseFinitePartIntegral.lean	Real integral representation of the finite part and endpoint justification.
GammaSecondOrder.lean	Second-order expansion of the Gamma factor near zero.
StieltjesConstant.lean	Chosen first Stieltjes constant convention and zeta expansion at one.
LaplaceCumulantAsymptotics.lean	Large-parameter cumulant expansions and derivative scale estimates.
LaplacePeriodicSecondOrder.lean	Second-order Laplace expansion retaining derivatives of the periodic correction.
EndpointLaplaceComparison.lean	Quantitative transfer from transform asymptotics to endpoint values of the Fabius function.
Lambert coordinate, saddle reduction, and sharp first-order theory	
LowerLambertW.lean	Construction and uniqueness of the lower real Lambert-W branch and the equation $n2^{-n} = x$.
FabiusLogScale.lean	Dyadic logarithmic coordinate and filter conversions between t to infinity and x to zero.
FabiusSmallArgumentScale.lean	Common small-argument neighborhoods, positivity, and asymptotic scale comparisons.
FabiusLogMainDefect.lean	Exact defect identities between logarithmic main terms and the transform-side expression.
FabiusLambertPhase.lean	Exact phase lambda, fixed-point equation, two-term expansion, and quantitative remainders.
FabiusLambertRates.lean	Big-O comparisons between inverse phase, inverse t, and inverse negative log x.
FabiusLambertDerivativeBounds.lean	Derivative estimates for the exact phase and its refined remainders.
FabiusLambertHigherExpansion.lean	Higher explicit terms needed to reach the literal O(1/t) elementary expansion.
FabiusLambertFormalLog.lean	Formal logarithmic algebra for higher Lambert-phase coefficients.
FabiusLambertSaddle.lean	Pairing of the exact phase with the saddle radius and stationarity identities.
FabiusLambertMinorArc.lean	Minor-arc estimates expressed uniformly in the exact Lambert coordinate.

Continued on next page

Module	Principal role (continued)
FabiusLambertTailFlat.lean	All-power flatness of the forward and contour tails in inverse phase.
BromwichSaddle.lean	Bromwich inversion setup, contour parameterization, and exact saddle decomposition.
QuantitativeSaddle.lean	Generic central-versus-tail quantitative saddle theorem.
FabiusBromwichInput.lean	Verification that the Fabius transform satisfies the generic Bromwich hypotheses.
FabiusSaddleReduction.lean	Normalization of the Fabius inversion integral to a dimensionless saddle ratio.
FabiusSaddleReferenceWeight.lean	Gaussian reference weight and normalization at the exact saddle.
FabiusSaddleReferenceTail.lean	Tail bounds for the reference Gaussian weight.
FabiusSaddleCentral.lean	Central-arc comparison with the reference weight at first quantitative order.
FabiusSaddleCentralLambert.lean	First-order normalized mass estimate in the exact Lambert coordinate.
FabiusSaddleCentralRadiusAsymptotics.lean	Asymptotic size and admissibility of the chosen central radius.
FabiusSaddleTail.lean	First-order bound for the noncentral contour contribution.
FabiusSharpConstant.lean	Exact constant assembled from Mellin, Gaussian, and scaling normalizations.
FabiusSharpLambertMain.lean	Compact exact-phase logarithmic saddle main term.
FabiusSharpLambertTransfer.lean	Transfer of sharp Lambert-coordinate estimates to the small-x filter.
FabiusExplicitSharpTransfer.lean	Conversion from compact Lambert form to the literal log/log-log expression.
FabiusSharpExactReduction.lean	Exact identity reducing log F minus the main term to log saddle ratio plus flat tail.
FabiusSharpAsymptoticTransfer.lean	Generic transfer from the normalized kernel-mass estimate to the sharp theorem.
FabiusFirstSaddleCorrection.lean	Closed first periodic coefficient beyond the sharp main term.
FabiusWikipediaMain.lean	Corrected compact Wikipedia main term and equality with the saddle main.
FabiusWikipediaExpansion.lean	Literal log x/log(-log x) expansion with exact-phase periodic correction.

Continued on next page

Module	Principal role (continued)
FabiusWikipediaObstruction.lean	Proof that omitting the nonconstant periodic correction destroys the claimed remainder.
FabiusSharpAsymptotic.lean	Public corrected O(1/(-log x)) theorem, negative obstruction, and exponentiated equivalence.
FabiusQuotientExponentialMismatch.lean	Endpoint-rate proof that a quotient-of-exponentials numerical fit is not asymptotically equivalent.
FabiusFlatness.lean	Vanishing to every algebraic order at the endpoint and related flatness theorems.
FabiusDecayComparison.lean	Faster-than-power and slower-than-exp(-c/x) comparison theorems.
FabiusLogSquaredAsymptotic.lean	Coarse quadratic-log asymptotic and O(t log t) control used in the draft audit.
All-orders saddle and Lambert expansion	
SaddleAllOrders.lean	Generic all-orders saddle theorem combining finite central expansions and flat tails.
SaddleExpansionAlgebra.lean	Coefficient convolution and finite asymptotic-series algebra.
SaddleExpansionFiniteRemainder.lean	Remainder bookkeeping for finite products and exponentials.
SaddleExpansionFlat.lean	Closure of asymptotic expansions under addition of functions flat to every order.
SaddleExpansionSmallArgument.lean	Transport of expansions from logarithmic infinity to a one-sided small-argument filter.
SaddleLogExpansionAlgebra.lean	Formal recurrence for logarithms of coefficient series with unit constant term.
SaddleLogExpansionPowerSeries.lean	Power-series realization and coefficient correctness for the logarithmic recurrence.
SaddleLogAsymptoticTransfer.lean	Generic passage from a positive ratio expansion to an expansion of its real logarithm.
GaussianPolynomialContraction.lean	Finite Gaussian moment contraction of polynomial exponent coefficients.
GaussianPolynomialWholeIntegral.lean	Exact whole-line Gaussian polynomial integrals.
GaussianPolynomialTail.lean	Quantitative tails for Gaussian polynomial weights.
GaussianPolynomialTailAllOrders.lean	Tail flatness uniformly at arbitrary asymptotic order.
FabiusSaddlePolynomialCoefficients.lean	Polynomial coefficient families obtained by exponentiating the vertical jet.

Continued on next page

Module	Principal role (continued)
FabiusSaddleExpansionCoefficients.lean	Normalized mass and log coefficient definitions and structural identities.
FabiusSaddleCoefficientRecurrence.lean	Finite recurrence computing successive saddle coefficients.
FabiusSaddleExponentAllOrders.lean	All-order expansion of the centered complex saddle exponent.
FabiusSaddleCentralAllOrders.lean	Arbitrary-order central-arc comparison and remainder.
FabiusSaddleTailAllOrders.lean	Flatness of the noncentral contour contribution to every inverse-phase order.
FabiusSaddleMassAllOrders.lean	Full normalized kernel-mass expansion in inverse exact Lambert phase.
FabiusLambertAllOrderAlgebra.lean	Formal algebra for arbitrary-order expansion of the lower-Lambert phase.
FabiusLambertAllOrderRemainder.lean	Quantitative remainder theorem for finite Lambert-phase expansions.
FabiusLambertAllOrderSmallArgument.lean	Transfer of all-order phase expansions to x tending to zero.
FabiusFullAsymptoticExpansion.lean	Full exact-phase expansion of $\log F$ and finite $O(\lambda^{-N})$ truncations.
Dyadic sharp asymptotics and specialized transfers	
DyadicSharpDecomposition.lean	Exact decomposition of dyadic logarithms into main, periodic, and residual terms.
DyadicSharpConditional.lean	Conditional implication from discrete kernel estimates to a dyadic sharp theorem.
FabiusDyadicSharpCumulant.lean	Cumulant estimates specialized to inverse-dyadic arguments.
FabiusDyadicSharpAsymptotic.lean	Unconditional sharp asymptotic along exact dyadic sequences.
Paper facades and audits	
Paper05442.lean	Numbered theorem facade for arXiv:1702.05442, with explicit source corrections.
Paper06487.lean	Numbered theorem facade for arXiv:1702.06487v3 and its repaired hypotheses/formulas.
PaperFabiusAsymptotic.lean	Aggregate for the corrected sharp theorem, all-orders expansion, and discrete/q-binomial asymptotics.
PaperKFoldThueMorse.lean	Claim-level audit of the local K-fold Thue–Morse draft, including proofs and refutations.
Auxiliary asymptotic estimates	

Continued on next page

Module	Principal role (continued)
`StirlingAsymptotics.lean`	Explicit Stirling estimates and asymptotic forms used in recurrence and draft-comparison arguments.

Root umbrella. `Analysis/FabiusFunction/Lean/FabiusFunction.lean` imports the paper facades and selected public branches, providing the one-line `import FabiusFunction` entry point.

B

Key declaration index

The catalogue in appendix A answers "which file should I open?" This appendix answers the complementary question "which declaration should I search for?" It is intentionally selective: each entry is either a public interface, a major bridge theorem, or a proof-engineering hinge on which a larger branch depends. Names are given exactly as they appear in the pinned source tree.

Declaration	Role in the development
Specifications and canonical objects	
`BoundedFabius`	Subtype of functions whose values remain in the unit interval; the invariant makes boundedness available without reproving it at every integral or norm estimate.
`IsFabius`	Abstract cumulative-function specification: the endpoint behavior, symmetry, and differential/refinement law used throughout the canonical branch.
`rvachevUp`	Piecewise folding of a bounded Fabius function into the even compactly supported bump on $[-1, 1]$.
`IsOriginalFabius`	Original compact-support characterization with its a priori unknown scaling constant; `IsOriginalFabius.mk_of_derivative_law` derives its positivity assumption.
`fabius`	Canonical bounded function obtained as the fixed point of the smoothing operator.
`fabius_spec`	Proof that the canonical fixed point satisfies `IsFabius`.
`fabius_unique`	Uniqueness of a bounded Fabius implementation, proved through equality on dyadic rationals and continuity.
`originalFabius_unique`	Uniqueness for the original compact-support formulation, proved by normalization and Fourier-transform injectivity.
Differential and global identities	
`rvachev_hasDerivAt`	The global refinement derivative for the up function, with the piecewise seams discharged explicitly.

Continued on next page

Declaration	Role in the development (continued)
`rvachev_contDiff`	Smoothness bootstrap: the derivative law raises finite differentiability one order at a time, yielding C^∞.
`extendedFabius`	Signed Thue–Morse translate sum that extends the bounded object to a global smooth function.
`extendedFabius_eq_fabiusReal`	Agreement of the signed global extension with the bounded Fabius coordinate on $[0,1]$.
`hasDerivAt_extendedFabius`	Global identity $\tilde{F}'(x) = 2\tilde{F}(2x)$, proved by local finiteness and translate reindexing.
`iteratedDeriv_extendedFabius`	Closed formula for every iterated derivative, exposing the triangular power of two.
`extendedFabius_add_pow_two`	Power-of-two translation negates the signed extension on the corresponding initial block.
`taylorRemainder_translate`	Exact cancellation of adjacent translated Taylor remainders.
`extendedFabius_reduction`	Finite recursive reduction formula obtained by combining Taylor's theorem, inverse-dyadic derivatives, and translated-remainder cancellation.
Exact arithmetic and dyadic evaluation	
`binaryWeight`	Number of 1 bits in a natural number; the parity controls the Thue–Morse sign.
`thueMorseSign`	The $\{\pm 1\}$ version of the Thue–Morse sequence used in all exact finite sums.
`moment`	Exact rational moment sequence defined before any analytic identification.
`halfMoment`	Exact rational half-moment sequence used at inverse powers of two and in the dyadic evaluator.
`fabiusDyadic`	Closed rational formula for a dyadic numerator/denominator pair.
`fabiusAtInverseTwoPow`	Exact value at 2^{-n}, packaged independently of a real-valued implementation.
`fabiusDyadicTable`	Finite table of inverse-power values consumed by the executable evaluator.
`evalDyadic`	Executable exact evaluator whose control flow clears binary blocks of the numerator.
`evalDyadic_eq_fabiusDyadic`	Arithmetic correctness theorem: the implementation computes the closed exact rational expression.
`fabiusDyadic_cast`	Principal exact-to-real bridge identifying the rational dyadic formula with any analytic Fabius implementation.

Continued on next page

Declaration	Role in the development (continued)
`fabiusDyadic_cast_extended`	Global signed-extension version of the dyadic cast theorem.
`halfMomentFabiusValue`	Identification of the exact half moment with the suitably normalized value at an inverse power of two.
Moments, transforms, and probability	
`momentPowerSeries`	Formal power series carrying the exact moment recurrence.
`halfMomentPowerSeries`	Formal series for the half moments and their functional equation.
`rvachevMoment`	Analytic integral moment of the compactly supported up function.
`rvachevMoment_eq_moment`	Bridge from the analytic integral to the exact rational moment sequence.
`rvachevFourier`	Complex Fourier transform of the up function.
`rvachevFourier_eq_product`	Infinite sinc-product formula obtained from the refinement equation.
`weightedCoordinateSum`	Random series $\sum_{n\geq 0} X_n/2^{n+1}$ on the infinite product of unit intervals.
`weightedSumDistribution_selfSimilar`	Distributional fixed-point identity after separating the first coordinate from the tail.
`weightedSumCDF_eq_integral`	Integral smoothing equation for the CDF of the random series.
`rvachev_poisson_summation`	Corrected Poisson summation formula, retaining the source's omitted phase variable.
`rvachevFourier_real_iteratedDeriv_rapidDecay`	Arbitrary polynomial decay for every real-axis derivative of the Fourier transform.
`rvachev_partition_unity`	Integer translates of the up function form a partition of unity.
q-binomial and discrete-limit machinery	
`halfQBinomial`	Gaussian binomial coefficient at $q = 1/2$, kept rational for exact coefficient extraction.
`halfQBinomial_theorem`	Finite q-binomial product identity underlying the Toeplitz row and scalar formulas.
`fabiusQBinomialFormula`	Exact q-binomial representation of Fabius values after the Thue–Morse generating-function reduction.
`fabiusDiscreteLimitRangeLength`	Safe natural-number encoding of the floor-defined inner prefix, including the empty-prefix case.

Continued on next page

Declaration	Role in the development (continued)
`discreteLimitWeight`	Outer finite-row weight obtained by reindexing the q-binomial approximant.
`sum_range_discreteLimitWeight`	Exact row-sum identity $\sum_j w_{n,j} = 1$.
`sum_abs_discreteLimitWeight_le`	Uniform total-variation bound needed for Toeplitz convergence.
`tendsto_weighted_rows_of_tendsto`	Generic finite-row Toeplitz theorem factored away from all Fabius-specific coefficients.
`fabiusUniformSpline`	Half-cell-centered finite Thue–Morse spline appearing in the discrete-limit representation.
`fabiusUniformSpline_block_translate`	Complete length-two block translation changes the spline by the corresponding Thue–Morse sign.
Legendre analysis	
`legendrePolynomial`	Generic Legendre polynomial implementation used by the spectral branch.
`legendre_orthogonal`	Sturm–Liouville orthogonality theorem on $[-1, 1]$.
`tendstoUniformly_legendreSeries`	Generic uniform convergence theorem under the smoothness hypotheses established in the infrastructure layer.
`fabiusLegendreCoefficient`	Exact coefficient functional for the up function, reduced to known moments.
`rvachev_eq_tsum_legendre`	Uniformly convergent Legendre expansion of the up function.
`legendrePartialSum_isLeastSquares`	Generic projection theorem identifying a finite Legendre sum as the least-squares polynomial approximant.
Negative Laplace product and periodic correction	
`negativeLaplaceKernel`	Logarithm of a single normalized factor $(1-e^{-x})/x$.
`negativeLaplaceLog`	Absolutely convergent logarithmic series for the canonical negative Laplace product.
`negativeLaplaceLog_two_mul`	Exact dyadic dilation equation for the logarithm.
`negativeLaplaceTailError`	Positive exponentially small forward-tail correction in the exact decomposition.
`negativeLaplacePeriodic`	One-periodic correction obtained by subtracting the quadratic solution of the dilation equation.
`negativeLaplacePsi`	Centered version of the periodic correction used in the final sharp main term.
`negativeLaplacePsi_nonconstant`	Proof that the periodic term cannot be absorbed into a constant.
Lambert phase and sharp asymptotics	

Continued on next page

Declaration	Role in the development (continued)
`lowerLambertW`	Lower real branch used to solve the small-argument saddle equation.
`fabiusLambertPhase`	Exact saddle coordinate $\lambda(x)$ satisfying $\lambda 2^{-\lambda} = x$.
`dyadicLambertPhase`	Same coordinate after substituting $x = 2^{-t}$.
`dyadicLambertPhase_fixedPoint`	Additive equation $\lambda - \log_2 \lambda = t$.
`fabiusCorrectedWikipediaMain`	Compact lower-Lambert main term with the centered periodic correction at the exact phase.
`fabiusExplicitCorrectedWikipediaMain`	Literal logarithmic expansion in $\log x$ and $\log(-\log x)$, still retaining the exact-phase periodic term.
`log_fabius_sub_explicitCorrectedWikipediaMain_isBigO`	Corrected sharp logarithmic theorem with error $O(1/(-\log x))$.
`log_fabius_sub_WikipediaElementaryMain_not_isBigO`	Obstruction theorem proving that the same rate is false when the periodic term is omitted.
`fabius_isEquivalent_exp_explicitCorrectedWikipediaMain`	Exponentiated asymptotic equivalent for the Fabius function itself.
All-orders saddle expansion	
`HasAsymptoticExpansion`	Generic coefficient/remainder interface used for arbitrary-order finite truncations.
`fabiusSaddleMassCoefficient`	Phase-dependent coefficient functions for the normalized complex saddle mass.
`fabiusSaddleLogCoefficient`	Coefficients after taking the logarithm of the normalized mass expansion.
`fabiusSaddleKernelMass_dyadicLambert_hasAsymptoticExpansion`	Full complex kernel-mass expansion on the dyadic logarithmic scale.
`log_fabius_dyadicReal_sub_sharpLambertMain_hasAsymptoticExpansion`	Full expansion of the logarithmic error relative to the exact sharp main term.
`fabiusSaddleLogPartialSum`	Finite sum of the first N phase-dependent logarithmic coefficients.
`fabiusSharpLambertExpansion`	Sharp main term augmented by a finite exact-phase saddle correction.
`fabiusSaddleLogPartialSum_two`	Identification of the first nonzero finite correction as `fabiusFirstSaddleCorrection`/λ.
`log_fabius_sub_sharpLambertExpansion_isBigO`	Remainder after N corrections is $O(\lambda(x)^{-N})$.
`log_fabius_sub_sharpLambertExpansion_isBigO_negLog`	Weakened but elementary rate $O((-\log x)^{-N})$, while coefficients remain at the exact Lambert phase.

Lean engineering

Several of these declarations have canonical specializations or equivalent "`HasSum` versus `tsum`" forms. Search by the semantic stem rather than assuming one exact suffix. Conversely, do not replace an exact rational declaration by its real cast too early: many denominator, parity, and evaluator theorems become substantially harder after coercion.

C

Source-claim and correction map

One of the development's most valuable features is that it does not silently massage source statements until Lean accepts them. It distinguishes literal claims, repaired claims, conditional reductions, and machine-checked refutations. The following map collects the most important examples discussed in the main text.

Source locus	Formal issue	Lean treatment
Arias de Reyna, first paper, finite convolution formula	Endpoint atoms cannot be represented by the same open-interval density used in the interior.	The approximant branch separates the atomic endpoints and proves the corresponding measure identity rather than hiding half-weights in informal notation.
First paper, approximation limits	A density-level convergence statement alone is insufficient at discontinuity or boundary points.	`StepApproximationLimit.lean`, `EarlyMeasureBridge.lean`, and `WeakConvergence.lean` isolate pointwise, measure, and characteristic-function limits with their proper hypotheses.
First paper, equation (25)	The printed exponential omits its dependence on the translation variable t.	`rvachev_poisson_summation` includes $\exp(2\pi ikt/u)$ and documents why the factor is forced by the Fourier coefficient calculation.
First paper, equation (32)	After multiplication by the scale parameter, a displayed factor $1/a$ should disappear.	`PoissonSummation.lean` states both the unmultiplied identity and the corrected multiplied form.
Original compact-support characterization	The scale constant is not initially known to be 2.	`OriginalCharacterization.lean` and `OriginalUniqueness.lean` first derive the scale from support/normalization data, then apply the Fourier argument.
Arithmetic paper, numbered statements	Some source ranges implicitly assume positive indices even though total Lean definitions include $n = 0$.	Facade theorems preserve the intended positive range; supplement theorems such as `theorem_nine_all` record the valid all-natural extension separately.
Arithmetic paper, proof of Theorem 20	A proof-internal sentence omits a leading factor 2 in the naturality claim for normalized half moments.	`two_mul_halfMoment_mul_reshetnikovMultiplier_isNatural` states and proves the corrected product, followed by the stronger odd-natural result.

Continued on next page

Source locus	Formal issue	Lean treatment (continued)
Arithmetic paper, dyadic formulas	Informal reordering of finite sums can obscure range obligations and coercions.	`Paper06487Supplement.lean` exposes both compact `fabiusDyadic`-based statements and expanded finite sums with explicit bounds.
Local K-fold Thue–Morse draft, equation (1)	The literal normalization is incompatible with exact low-order calculations.	`DraftCounterexamples.lean` gives concrete machine-checked counterexamples; `ThueMorseApproximation.lean` states a corrected pointwise theorem.
Same draft, local/global error claims	The claimed bounds rely on the false normalization and do not survive exact evaluation.	The aggregate `PaperKFoldThueMorse.lean` exports refutations rather than weakening the claims without notice.
Same draft, equation (7)	The asserted maximizer is false.	A dedicated counterexample theorem identifies the failure; the lower-Lambert analysis is developed independently and is not presented as a repair of the false maximum claim.
Same draft, equation (10)	The proposed proxy misses a linear term.	The mismatch is proved exactly, and the proxy is excluded from the rigorous asymptotic chain.
Wikipedia-style elementary asymptotic	A nonconstant log-periodic term is omitted.	`FabiusWikipediaObstruction.lean` proves the omitted term is genuine; `FabiusSharpAsymptotic.lean` adds `negativeLaplacePsi` at the exact lower-Lambert phase and proves the corrected rate.
Replacing the exact Lambert phase by $t + \log_2 t$ too early	The phase error is amplified by differentiation of the quadratic main term and can lose a logarithmic factor.	`FabiusLambertHigherExpansion.lean` supplies the needed refined phase terms; final periodic coefficients remain evaluated at the exact phase.
Quotient-of-exponentials numerical fit	A close compact-interval fit may be mistaken for an endpoint asymptotic equivalent.	`FabiusQuotientExponentialMismatch.lean` proves that its decay scale is incompatible with the Fabius endpoint behavior.
All-orders "elementary" expansion	Substituting an elementary phase series into every oscillatory coefficient requires a nontrivial composition theorem.	`FabiusFullAsymptoticExpansion.lean` claims the full expansion at the exact phase and explicitly declines the stronger unproved composition claim.

Formalization-driven correction

The correction modules are not peripheral commentary. They are part of the public mathematical result: a failed source statement is replaced by a pair of kernel-checked facts—a precise obstruction and the strongest corrected statement actually supported by the proof. This makes later reuse safer than a prose erratum whose downstream consequences have not been rechecked.

D

Reproduction and audit checklist

This appendix gives a compact workflow for reproducing the source snapshot, building the focused library, and checking the architectural claims made in this walkthrough. The commands are intentionally ordinary shell commands; none depends on editor integration.

D.1 Clone and pin

```
git clone https://github.com/VladimirReshetnikov/ProveIt.git
cd ProveIt
git checkout 40c0d637d310a9f16d96f53d287438b7d9b1f53f
git status --short --branch
cat lean-toolchain
```

The final two commands should show a detached checkout at the pinned commit and the Lean 4.32.0 toolchain. Keeping the checkout clean matters when comparing module counts or generated dependency information.

D.2 Build the focused library

```
lake update
lake build FabiusFunction
```

A successful `lake build FabiusFunction` is the decisive compilation check. It causes Lean to elaborate the complete import closure of the named library and lets the kernel check the generated proof terms. Building only a single facade file can miss an unimported branch.

For a narrow edit loop, compile a chosen module through Lake so the package roots and mathlib environment remain correct:

```
lake env lean Analysis/FabiusFunction/Lean/FabiusFunction/DyadicAnalytic.lean
```

D.3 Enumerate the source tree

```
find Analysis/FabiusFunction/Lean -name '*.lean' -type f | sort
find Analysis/FabiusFunction/Lean/FabiusFunction \
  -maxdepth 1 -name '*.lean' -type f | wc -l
```

At the pinned snapshot, the second command counts 164 leaf modules; the root `Analysis/FabiusFunction/Lean/FabiusFunction.lean` is the separate umbrella. This tree count is why the catalogue in appendix A is larger than the older scripted count recorded in the repository's audit note.

D.4 Inspect imports rather than guessing dependencies

For a single module, its direct dependencies are visible at the top of the file:

```
sed -n '1,40p' \
  Analysis/FabiusFunction/Lean/FabiusFunction/FabiusSharpAsymptotic.lean
```

For a lightweight textual graph, extract all local imports:

```
grep -Rho '^import FabiusFunction\.[A-Za-z0-9_]*' \
  Analysis/FabiusFunction/Lean | sort -u
```

Textual imports are not a substitute for Lean's environment, but they are excellent for locating layer violations—for example, an exact arithmetic file unexpectedly importing a real-analysis facade.

D.5 Interrogate theorem dependencies in Lean

For a theorem whose trust surface matters, create a small scratch file at the repository root:

```
import FabiusFunction

#print axioms Fabius.fabius_unique
#print axioms Fabius.evalDyadic_eq_fabiusDyadic
#print axioms Fabius.log_fabius_sub_explicitCorrectedWikipediaMain_isBigO
```

Then run it with `lake env lean Scratch.lean`. The output reports the axioms on which the elaborated declaration depends. In a classical mathlib development, appearances of `propext`, `Classical.choice`, and `Quot.sound` are normal; the important check is that no unreviewed project-specific axiom has entered the chain.

D.6 Search for placeholders with context

A raw search is useful, but comments and documentation may mention the word "sorry." Inspect matches rather than treating the count as a proof:

```
grep -RIn --include='*.lean' -E '\bsorry\b|\badmit\b' \
  Analysis/FabiusFunction/Lean
```

The authoritative check remains successful elaboration plus theorem-level `##print axioms` inspection. A source grep cannot detect every possible trust extension, and it can report harmless prose.

D.7 Recompute an exact dyadic value

A useful smoke test exercises both computation and the analytic cast. In a scratch file, evaluate a rational term and ask Lean to simplify a concrete instance:

```
import FabiusFunction

#eval Fabius.evalDyadic 8 37
#check Fabius.evalDyadic_eq_fabiusDyadic
#check Fabius.fabiusDyadic_cast
```

The exact printed value is less important than the chain being exercised: executable recursion, closed rational specification, and the theorem that casts the result into the analytic function.

D.8 Read the repository's own coverage documents

The two most useful prose companions are:

- `Analysis/FabiusFunction/README.md`, for the broad inventory and build entry point;
- `Analysis/FabiusFunction/docs/PAPER_COVERAGE.md`, for the numbered-claim matrix and recorded source corrections;
- `Analysis/FabiusFunction/docs/ASYMPTOTIC_COMPLETION_AUDIT.md`, for the completion status of the sharp and all-orders asymptotic tower.

These documents are navigation aids. The Lean declarations remain the source of truth, especially where the source tree has grown after an audit count was recorded.

Lean engineering

A good review order is *compile, inspect the exact declaration, inspect its imports, print its axioms, then read the facade prose.* Starting from a paper-facing theorem statement alone can conceal whether the real work is arithmetic normalization, measure transport, a generic convergence lemma, or a quantitative contour estimate several layers below.

E

Notation and Lean translation guide

The same mathematical object frequently appears under several Lean types. The following table summarizes the most common translations used in the code and in this walkthrough.

Mathematical role	Typical Lean representation	Why that representation is used
Bounded cumulative function	`BoundedFabius`	Carries $0 \le F(x) \le 1$ in the type, making bounded convergence and norm estimates local.
Abstract Fabius law	`IsFabiusF`	Separates downstream theorems from the particular fixed-point construction.
Compact bump	`rvachevUpF:ℝ→ℝ`	Symmetric support and global differential refinement are cleaner in the up coordinate.
Signed global function	`extendedFabiusF:ℝ→ℝ`	Dyadic translation and derivative iteration become algebraic rather than piecewise.
Exact value	$\mathbb{Q}$	Preserves denominators, parity, divisibility, and executable normalization.
Dyadic grid point	numerator and exponent in $\mathbb{N}$	Avoids quotient representations and aligns induction with binary control flow.
Integer dyadic scale	$\mathbb{Z}$ exponent and `zpow`	Represents both large and small powers of two without case-specific reciprocal notation.
Finite exact sum	`Finset.sum` or a sum over `Finn`	Exposes range obligations and lets normalization tactics close concrete arithmetic.
Locally finite global sum	`tsum` plus support lemmas	Gives a convenient global definition while proofs reduce every point to a finite sum.
Moment generating identity	`FormalMultilinearSeries` or `PowerSeries`	Coefficient algebra is proved before convergence and integration enter.
Compact smooth transform input	`SchwartzMapℝℂ`	Unlocks mathlib's Fourier injectivity and Poisson summation APIs.

Mathematical role	Typical Lean representation	Why that representation is used (continued)
Random series law	`Measureℝ` and `cdf`	Makes self-similarity, independence, and weak convergence explicit.
Uniform function convergence	`Tendsto` in a sup-norm or uniform topology	Supports termwise limiting and pointwise corollaries without informal interchange.
Asymptotic bound	`IsBigO`, `IsLittleO`, `IsEquivalent`	Records the filter and scale explicitly, especially at $x \to 0^+$.
All-orders expansion	`HasAsymptoticExpansion`	Packages coefficient growth and every finite remainder estimate into one reusable interface.
Periodic asymptotic coefficient	function of the exact Lambert phase	Retains the genuine dyadic oscillation instead of incorrectly replacing it by a constant.

A final architectural summary

The development succeeds because it repeatedly chooses the representation in which the next theorem is exact.

1. Binary and denominator questions are solved in $\mathbb{Q}$ before casting.
2. Smoothness is solved in the compact up coordinate before globalizing.
3. Translation identities are solved in the signed Thue–Morse extension.
4. Coefficient identities are solved in formal power series before analytic convergence is invoked.
5. Probability statements are solved at the level of product measures and maps before identifying the CDF with the canonical function.
6. Spectral convergence is solved generically before exact Fabius coefficients are substituted.
7. The endpoint asymptotic is solved first as an exact Laplace-product and saddle identity, with the periodic term and lower-Lambert phase kept intact; only then is it expanded into elementary logarithms.

That discipline also explains the apparent size of the library. The many modules are not gratuitous fragmentation. They mark changes of semantic coordinate and isolate the exact theorem that justifies each change. As a result, the public paper facades can remain short, executable formulas can be kernel-certified against analytic objects, and corrections to informal source claims can be stated without weakening the surrounding mathematics.

Source notes

This walkthrough was prepared from the pinned repository snapshot identified on the title page. The principal external mathematical sources mirrored by the facade modules are:

- Juan Arias de Reyna, *An infinitely differentiable function with compact support: definition and properties*, arXiv:1702.05442;
- Juan Arias de Reyna, *Arithmetic of the Fabius function*, arXiv:1702.06487v3;
- the local K-fold signed Thue–Morse draft tracked by `PaperKFoldThueMorse.lean`;
- the repository's README, paper-coverage matrix, and asymptotic completion audit.

The document deliberately uses declaration names and module boundaries from the Lean source rather than attempting to reproduce the papers' notation verbatim. Where a source formula is corrected, the relevant Lean module and obstruction theorem are identified in appendix C.

Pinned to `40c0d637d310a9f16d96f53d287438b7d9b1f53f`.

Generated as a human-readable companion to the kernel-checked Lean development.